# VERANTWOORDINGSVERSLAG

Paintshop scheduling; OR5



Fee van Zoghel, 5835704
Matthijs van Meeuwen, 5797799

Docent: Roy Willemen & Lonneke Driessen
Groep 14

# Inhoudsopgave

1 Inleiding ..... 3
2 Ontwerp van de oplosmethoden ..... 4
2.1 Representatie en doelfunctie ..... 4
2.2 Greedy constructive heuristic ..... 6
2.2.1 Greedy rule ..... 6
2.2.2 Pseudocode ..... 8
2.3 Discrete Improving Search ..... 10
2.3.1 Buurruimtestructuur ..... 10
2.3.2 Zoekstrategie ..... 11
2.3.3 Stopconditie en lokale optimaliteit ..... 11
2.3.4 Pseudocode improving search ..... 12
2.4 Metaheuristiek ..... 13
2.4.1Werking van de metaheuristiek ..... 13
2.4.2 Parameters en stopconditie ..... 14
2.4.3 Pseudocode ..... 15
2.5 Theoretische tijdscomplexiteit ..... 17
3 Implementatie van de oplosmethoden ..... 21
3.1 Opbouw van het Python-programma ..... 21
3.2 Overeenkomst tussen ontwerp en implementatie ..... 22
3.3 Invoer, uitvoer en generieke toepasbaarheid ..... 23
3.4 Visualisatie van planningen ..... 25
3.5 Leesbaarheid en documentatie van de code ..... 25
4 Validatie ..... 27
4.1 Testopzet ..... 27
4.2 Uitgevoerde tests en resultaten ..... 27
4.3 Validatie van lokale optimaliteit ..... 30
4.4 Conclusie van de validatie ..... 30
5 Experimenten ..... 31
5.1 Experimentele opzet ..... 31
5.2 Onderzoek naar parameterinstellingen ..... 32
5.3 Vergelijking van de oplosmethoden ..... 41
5.3.1 Experimentele opzet ..... 41

5.3.2 Vergelijking van de oplossingskwaliteit ..... 41
5.3.3 Vergelijking van de rekentijd ..... 43
5.3.4 Toegevoegde waarden van Discrete Improving Search ..... 43
5.3.5 Conclusie ..... 44
5.4 Keuze van oplosmethode en planning ..... 44
6 Eigen onderzoeksvraag of ontwikkelrichting ..... 46
6.1 Onderzoeksvraag of ontwikkelrichting ..... 46
6.2 Aanpak ..... 46
6.3 Resultaten ..... 48
6.4 Conclusie en toegevoegde waarde ..... 53
7 Conclusie ..... 54
References ..... 56
Bijlage A: Documentatie gebruik AI ..... 0
Bijlage B: Checklist Python Code ..... 0

# 1 Inleiding

Een verfspuiterij moet een aantal verfopdrachten inplannen op drie machines. Elke order heeft een oppervlakte, kleur, deadline en een boete per tijdseenheid bij overschrijding van de deadline. De machines verschillen in snelheid en bij een kleurwissel is een wisseltijd nodig, die per machine en per kleurwissel kan verschillen.

Het doel van de planning is:

*De totale penalty door te late orders te minimaliseren*

Het doel van deze programmeeropdracht is niet alleen een planning met een lage penalty te vinden, maar ook generieke en betrouwbare oplossingsmethoden te ontwikkelen die op verschillende datasets met andere opdrachten of machines kunnen worden toegepast, en deze te implementeren, valideren en evalueren. Hiervoor hebben we twee oplossingsmethoden ontwikkeld:

- Bij de eerste oplossingsmethode gebruiken we een EDD-heuristiek (Earliest Due Date), waarbij de Best Improvement-zoekstrategie wordt toegepast.
- Bij de tweede oplossingsmethode gebruiken we Simulated Annealing (SA), eveneens gevolgd door een Improving Search gebaseerd op swaps, waarbij de Best Improvement-zoekstrategie wordt toegepast.

Als aanvullend eigen onderzoek onderzoeken we welke combinatie van machinesnelheidsverhogingen nodig is om een totale penalty van 0 te bereiken, en vergelijken we hierbij beide oplossingsmethoden.

Dit verslag beschrijft eerst het ontwerp en de implementatie van de methoden, daarna de validatie en de experimenten waarmee we de methoden vergelijken, en sluit af met een kort onderzoek naar de machinesnelheden gevolgd door een conclusie.

# 2 Ontwerp van de oplosmethoden

In dit hoofdstuk beschrijven we en verantwoorden we het ontwerp van de oplossingsmethoden. We maken duidelijk welke keuzes wij hebben gemaakt, hoe de algoritmen werken en waarom wij deze keuzes passend vinden voor het planningsprobleem

## 2.1 Representatie en doelfunctie

Bij beide oplossingsmethoden wordt een oplossing weergegeven als een bepaalde volgorde van orders. In het programma wordt deze volgorde opgeslagen in een lijst met ordernummers. Op basis van deze volgorde worden de orders over de beschikbare machines verdeeld.

Iedere order wordt toegewezen aan de machine die het eerst beschikbaar is. Als meerdere machines op hetzelfde moment beschikbaar zijn, wordt de machine met de hoogste verwerkingssnelheid gekozen. Vervolgens worden de eventuele omsteltijd (setup time), de productietijd, de begintijd en de eindtijd van de order berekend. Hierdoor ontstaat een planning met een reeks orders per machine. Deze planning wordt uiteindelijk weergegeven in een Gantt-chart.

De kwaliteit van een oplossing wordt bepaald door de totale penalty. Wanneer een order na de deadline wordt afgerond, ontstaat er tardiness. Dit is de hoeveelheid tijd waarmee de deadline wordt overschreden.

De tardiness voor order (i) wordt berekend door:

$$T_i = max(0, E_i - D_i)$$

Hiervoor geldt dat:

$i = 1 \dots n; hierbij\ is\ i\ het\ ordernummer\ en\ n\ het\ totale\ aantal\ orders$
$T_i = tardiness\ van\ order\ i$
$E_i = eindtijd\ order\ i$
$D_i = deadline\ van\ order\ i$

Doelfunctie:

$$min: \sum_{i=1}^{n} T_i * P_i$$

Hiervoor geldt dat:

$$P_i = penalty\ van\ order\ i\ per\ tijdseendheid$$

Het doel van beide oplosmethoden is om de totale penalty te minimaliseren. Hoe lager de totale penalty, hoe beter de gevonden oplossing.

## 2.2 Greedy constructive heuristic

De Greedy Constructive Heuristic begint met het sorteren van alle orders volgens de EDD-regel, waarbij de order met de vroegste deadline als eerste wordt ingepland. Vervolgens worden de orders één voor één toegewezen aan de machine met de laagste huidige eindtijd. Wanneer meerdere machines dezelfde eindtijd hebben, wordt de snelste machine gekozen. Bij het toewijzen wordt rekening gehouden met een eventuele kleurwissel en de bijbehorende setuptijd. Daarna wordt de productietijd bepaald en wordt de eindtijd van de order berekend. Aan de hand van de eindtijd en deadline worden de tardiness en bijbehorende penalty bepaald. Vervolgens worden de machinetijd en de laatst uitgevoerde kleur bijgewerkt. Dit proces wordt voor iedere order herhaald totdat alle orders aan een machine zijn toegewezen en een volledige planning is ontstaan.

### 2.2.1 Greedy rule

Voor het maken van de planning hebben we verschillende greedy rules overwogen: EDD (Earliest Due Date) en SPT (Shortest Processing Time). Daarnaast hebben we gekeken naar een methode waarbij iedere machine zoveel mogelijk één kleur verwerkt.

Bij SPT worden de orders met de kortste verwerkingstijd als eerste ingepland. Hierdoor worden korte orders snel afgerond, kunnen er in korte tijd veel orders worden uitgevoerd en is de kans op een penalty voor korte orders klein. Een nadeel is dat lange orders steeds verder naar achteren worden geschoven, waardoor de totale penalty van deze lange orders hoger kan worden. Daarnaast wordt er niet gekeken naar de deadline en kan er veel tijd verloren gaan door het wisselen van kleur.

Bij deze greedy rule wordt iedere machine toegewezen aan één kleur. Hierdoor is er bijna geen wisseltijd tussen verschillende kleuren en blijft de planning eenvoudig. Een nadeel is dat er niet wordt gekeken naar de deadlines van de orders. Daarnaast kan het aantal orders per kleur ongelijk verdeeld zijn, waardoor de ene machine veel werk moet uitvoeren terwijl een andere machine weinig werk heeft.

Bij EDD worden de orders met de vroegste deadline als eerste ingepland. Hierdoor wordt er direct rekening gehouden met de deadlines en sluit deze methode goed aan bij het doel om de totale penalty te minimaliseren. Een nadeel is dat een order met een vroege deadline maar een lage penalty voorrang kan krijgen op een order met een iets latere deadline maar een hoge penalty. Daarnaast wordt bij het sorteren niet direct rekening gehouden met kleurwissels en de bijbehorende setup time.

Als we de verschillende greedy rules naast elkaar zetten, zien we dat EDD als enige direct rekening houdt met de deadlines van de orders. Bij SPT bestaat het risico dat lange orders

steeds naar achteren worden geschoven. Bij het verdelen van één kleur per machine kan daarnaast een ongelijke verdeling van het werk ontstaan. EDD geeft daarentegen prioriteit aan de orders die het eerst klaar moeten zijn. De deadlines bepalen uiteindelijk mede of een order te laat is en hoeveel penaltykosten hierdoor ontstaan. Omdat het minimaliseren van de totale penalty de doelfunctie van het probleem is, hebben we ervoor gekozen om EDD te gebruiken. Daarnaast hadden we tijdens de lessen en challenges al code voor EDD gemaakt. Deze code konden we als basis gebruiken en vervolgens verder aanpassen aan het planningsprobleem.

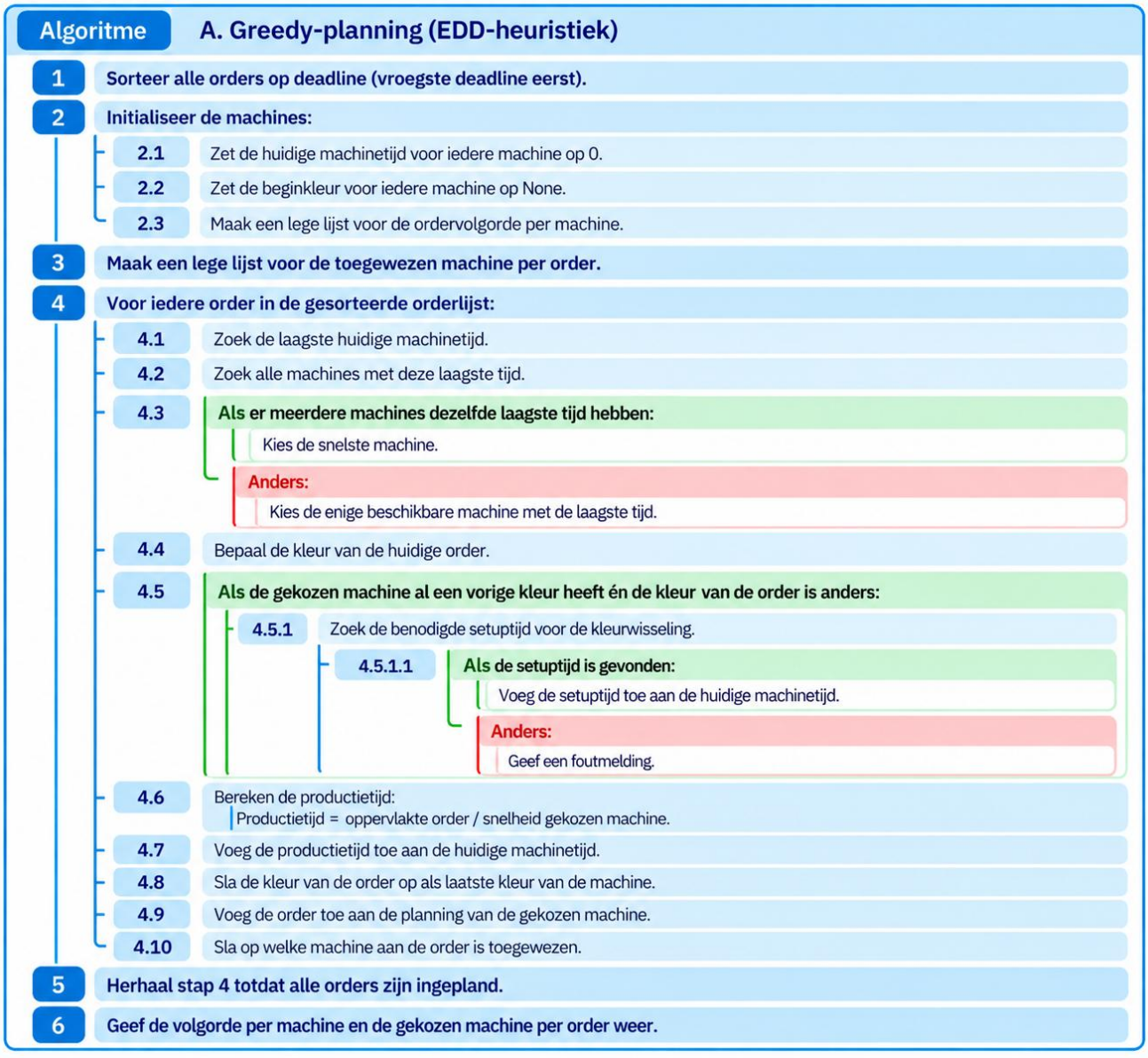
## 2.2.2 Pseudocode

**Greedy heuristic – EDD**

De Greedy Heuristic bestaat uit twee delen. In deel A wordt met behulp van de EDD-regel (Earliest Due Date) een planning opgesteld. Vervolgens worden in deel B de resultaten van deze planning berekend, waaronder de totale tardiness en de totale penalty.

**Figuur 2.1**

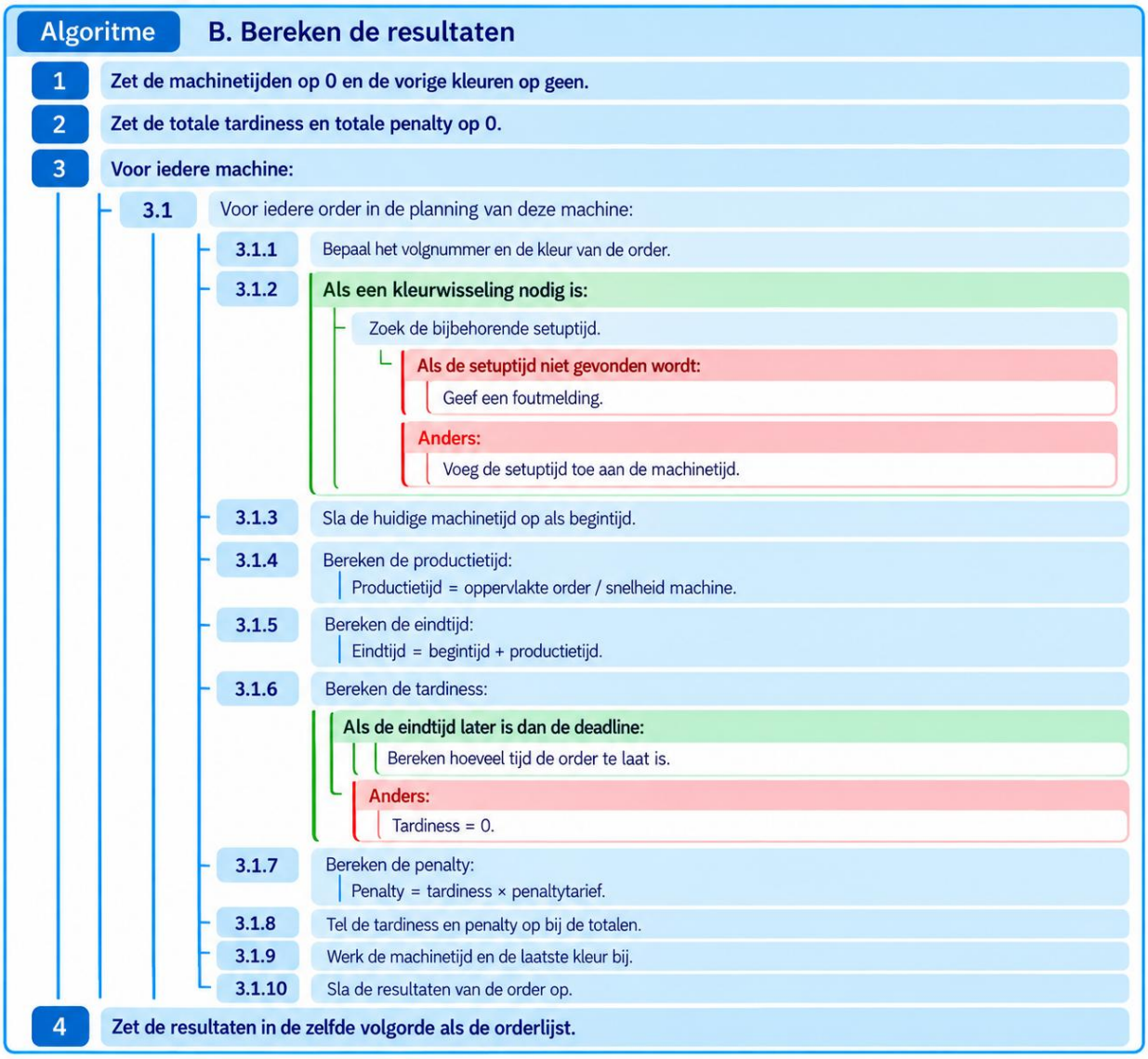
*Deel A, Greedy-planning*




Note: The pseudocode was developed by the authors. ChatGPT (OpenAI, 2026) was used to create the visual layout.

**Figuur 2.2**

*Berekening van de resultaten van de greedy-planning*




Note: The pseudocode was developed by the authors. ChatGPT (OpenAI, 2026) was used to create the visual layout.

## 2.3 Discrete Improving Search

De Improving Search gebruikt de planning uit de Greedy Heuristic als beginoplossing. Vanuit deze oplossing worden systematisch alle mogelijke swaps tussen twee orders onderzocht. Voor iedere swap wordt een nieuwe planning gemaakt en wordt de totale penalty berekend. Met behulp van Best Improvement wordt van alle onderzochte neighbour-oplossingen de oplossing met de laagste penalty gekozen. Wanneer deze penalty lager is dan die van de huidige oplossing, wordt deze neighbour-oplossing de nieuwe beste oplossing. Vanuit deze nieuwe oplossing worden vervolgens opnieuw alle mogelijke swaps onderzocht. Dit proces wordt herhaald totdat geen enkele swap meer een lagere totale penalty oplevert. Op dat moment is een lokaal optimum binnen de gekozen swap-neighbourhood bereikt.

### 2.3.1 Buurruimtestructuur

In de les zijn de keuzes flip en swap behandeld. Een voordeel van een flip is dat meerdere opeenvolgende orders in 1 flip van volgorde veranderen. Hierdoor ontstaat een grotere verandering in de planning dan bij een eenvoudige swap, waardoor andere delen van de oplossingsruimte kunnen worden onderzocht. Het nadeel hieraan is, dat het, omdat het meerdere orders tegelijkertijd van positie veranderd, er een goede gedeeltelijke volgorde ook weer slechter kan worden. Een flip levert niet automatisch een betere oplossing op (Zhang et al. (2025)).

De voordelen van een swap methode zijn dat ze eenvoudig uit te voeren en te programmeren zijn. Er gebeuren iedere keer kleine veranderingen waardoor je geen betere oplossingen over slaat. Veel verschillende buuroplossingen: bij n orders zijn er $\mathrm{n}(\mathrm{n}-1) * \frac{1}{2}$ verschillende pairwise swaps. Daardoor kun je systematisch een duidelijke neighbourhood onderzoeken (Idzikowski et al. (2021)). Nadelen hiervan zijn dat de veranderingen relatief klein zijn. Hierdoor kan de zoekmethode vastlopen in een lokaal optimum zonder het globale optimum te bereiken (Dong et al. (2015)). Het aantal oplossingen binnen de swap-neighbourhood groeit met $O(n^2)$, waardoor de rekentijd kan toenemen als het aantal orders toeneemt.

Omdat het gegeven probleem niet extreem groot is, is het mogelijk om alle swaps binnen de neighbourhood te onderzoeken zonder dat de rekentijd een te groot probleem wordt. Daarnaast blijft bij een swap een groot gedeelte van de oorspronkelijke planning behouden en kunnen alle buurruimtes systematisch worden gecontroleerd. Hierdoor kan duidelijk worden vastgesteld wanneer er binnen de gekozen neighbourhood geen verdere verbetering meer mogelijk is. Om deze redenen hebben we gekozen voor de swap-methode.

### 2.3.2 Zoekstrategie

Voor het zoeken naar een verbeterde oplossing is er gebruik gemaakt van best improvement. Vanuit de huidige oplossing worden alle mogelijke swaps tussen 2 orders onderzocht. Voor iedere swap wordt er een nieuwe planning gemaakt. Voor deze planning wordt de totale penalty berekend. Nadat alle swaps zijn onderzocht wordt de beste neighbour-oplossing gekozen (degene met de laagste totale penalty). Vanuit deze nieuwe oplossing worden opnieuw alle mogelijke swaps onderzocht. De swaps worden systematisch op volgorde onderzocht. Eerst wordt de eerste order verwisseld met iedere order die daarna komt, vervolgens de tweede order met iedere order die daarna komt, enz. totdat alle mogelijke combinaties zijn onderzocht.

Wanneer de totale penalty van de beste neighbour-oplossing lager is dan die van de huidige oplossing, wordt deze neighbour-oplossing de nieuwe incumbent solution. Vanuit deze oplossing worden opnieuw alle mogelijke swaps onderzocht. Dit proces wordt herhaald net zolang tot de incumbent solution geen neighbour meer heeft waarbij de totale penalty lager is. Op dit moment is er een lokaal optimum bereikt. Binnen de gekozen swap-neighbourhood is er dus geen verbetering die kan worden gevonden met dezelfde zoekmethode.

### 2.3.3 Stopconditie en lokale optimaliteit

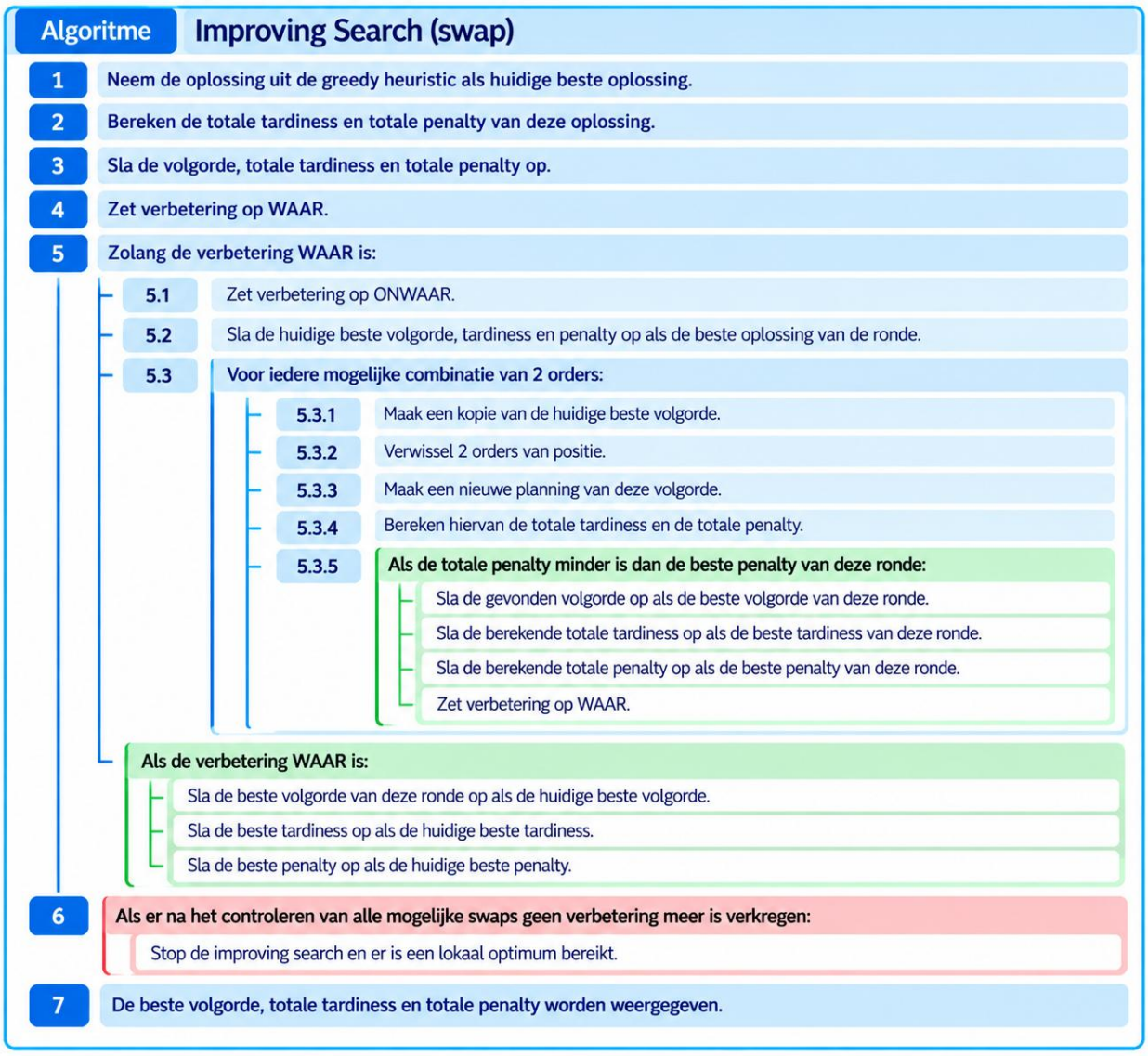
Vanuit de huidige oplossing worden alle mogelijke swaps onderzocht. Deze oplossing heeft een neighbourhood die bestaat uit alle mogelijke oplossingen die ontstaan door twee orders te swappen. Iedere neighbour heeft een eigen tardiness en penalty, die beter of slechter kan zijn dan die van de huidige oplossing. Uit alle neighbours wordt de oplossing met de beste verbetering gekozen, oftewel de oplossing waarbij de penalty het meest daalt. Dit proces wordt telkens herhaald. Op een gegeven moment wordt een huidige oplossing bereikt waarbij het swappen van twee orders geen betere oplossing meer oplevert. Geen enkele mogelijke combinatie van twee orders geeft dan nog een lagere penalty. Op dat moment stopt de Discrete Improving Search. Er is dan een lokaal optimum bereikt ten opzichte van de gekozen swap-neighbourhood.

### 2.3.4 Pseudocode improving search

De Improving Search gebruikt de oplossing van de Greedy Heuristic als startpunt en probeert deze te verbeteren door telkens twee orders van positie te wisselen (swap). Dit proces wordt herhaald totdat geen verbetering meer mogelijk is en een lokaal optimum is bereikt.

**Figuur 2.3**

*Improving search gebaseerd op swaps*




Note: The pseudocode was developed by the authors. ChatGPT (OpenAI, 2026) was used to create the visual layout.

## 2.4 Metaheuristiek

In deze paragraaf beschrijven we het ontwerp van de gekozen metaheuristiek en verantwoorden wij de belangrijkste ontwerpkeuzes

### 2.4.1Werking van de metaheuristiek

Als metaheuristiek gebruiken we Simulated Annealing (SA). SA start met een willekeurige ordervolgorde. Vanuit de huidige oplossing wordt telkens een kandidaatoplossing gemaakt door willekeurig twee orders te swappen. Voor iedere kandidaatoplossing wordt vervolgens een nieuwe planning gemaakt. De orders worden over de machines verdeeld op dezelfde manier als bij onze greedy heuristic. Per order wordt gekeken welke machine de laagste huidige eindtijd heeft. Als meerdere machines dezelfde eindtijd hebben, wordt de snelste machine gekozen. Hierdoor worden de orders tijdens het maken van de planning over de machines verdeeld en wordt voorkomen dat één machine een zeer groot deel van de orders krijgt toegewezen terwijl andere machines weinig orders krijgen.

Een kandidaat met een lagere penalty wordt altijd geaccepteerd. Een slechtere kandidaat kan worden geaccepteerd volgens de kans:

$$Kans = e^{\frac{a-b}{c}}$$
$$a = penalty\ van\ de\ huidige\ oplossing$$
$$b = penalty\ van\ de\ kandidaatoplossing$$
$$c = temperatuur$$

Hierdoor kan SA een lokaal optimum verlaten en andere oplossingen onderzoeken. De temperatuur wordt gedurende het proces verlaagd, waardoor slechtere oplossingen steeds minder vaak worden geaccepteerd. Wordt een kandidaat niet geaccepteerd dan blijft de huidige oplossing behouden.

Uiteindelijk hebben we gekozen voor de swap-move. Naast swap hebben we ook de flip-move overwogen. De onderbouwing voor de keuze van swap staat beschreven in paragraaf 2.3.1. Binnen onze SA wordt daarom een swap-neighbourhood gebruikt, waarbij willekeurig twee orders van positie worden gewisseld.

### 2.4.2 Parameters en stopconditie

t_max = maximaal aantal iteraties dat de SA uitvoert.

temp = begintemperatuur, deze beïnvloedt de kans dat er een slechtere oplossing wordt geaccepteerd.

cooling_factor = factor waarmee de temperatuur wordt verlaagd.

cooling_it = na iedere x aantal iteraties wordt de temperatuur met de cooling factor vermenigvuldigd.

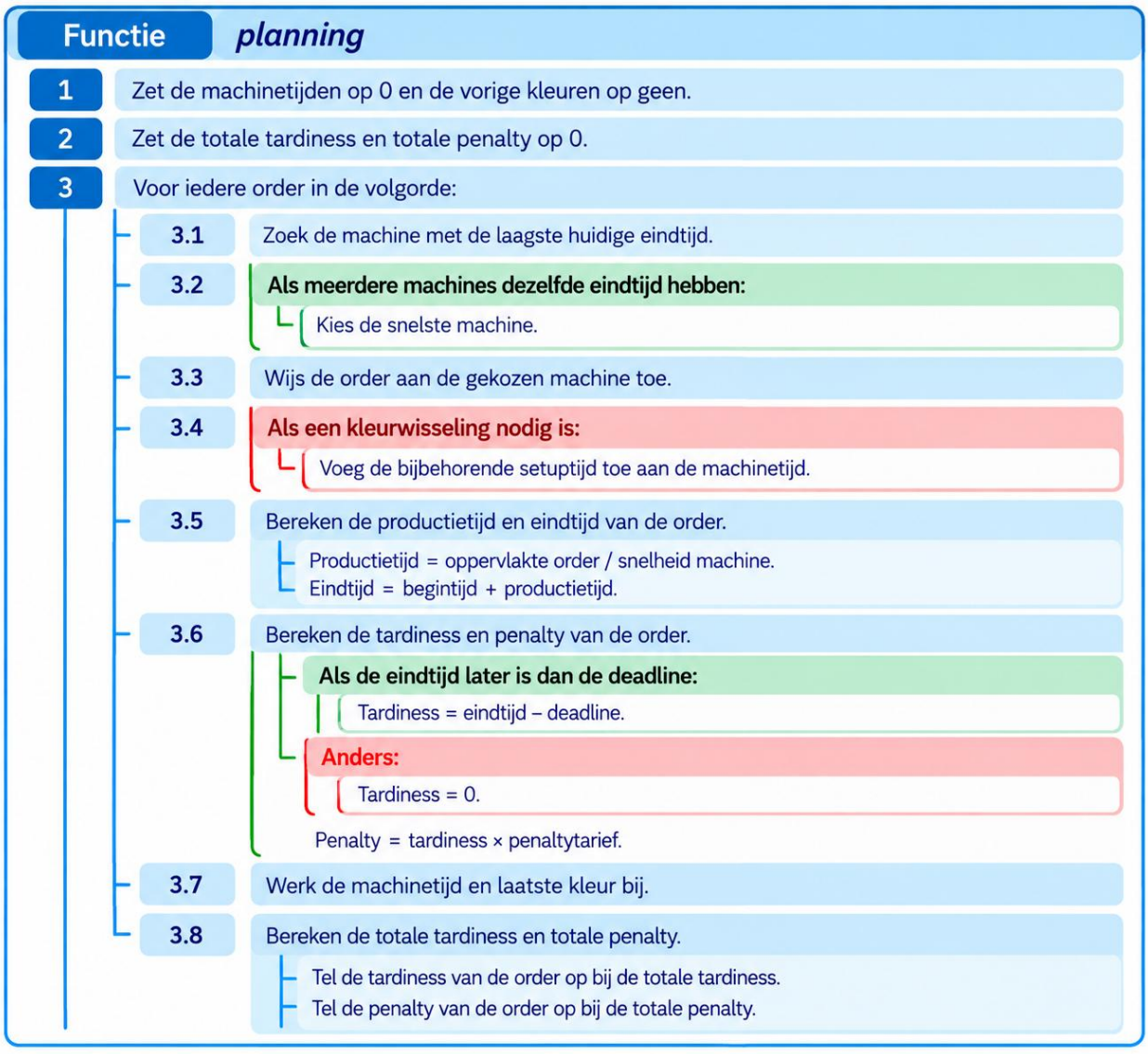
Voor SA zijn er vier parameters, die hierboven zijn genoemd en uitgelegd. Na iedere cooling_it iteraties wordt de temperatuur verlaagd met de ingestelde cooling_factor. Hierdoor verandert gedurende het proces de kans dat een slechtere oplossing wordt geaccepteerd. Als stopconditie wordt t_max gebruikt. Zodra het maximale aantal iteraties is bereikt, stopt SA en wordt de beste gevonden oplossing teruggegeven.

### 2.4.3 Pseudocode

Wanneer in de pseudocode een definitie wordt gebruikt wordt deze eerst afzonderlijk uitgewerkt. Vervolgens wordt er in de pseudocode cursief naar verwezen zoals bij *planning*.

**Figuur 2.4**

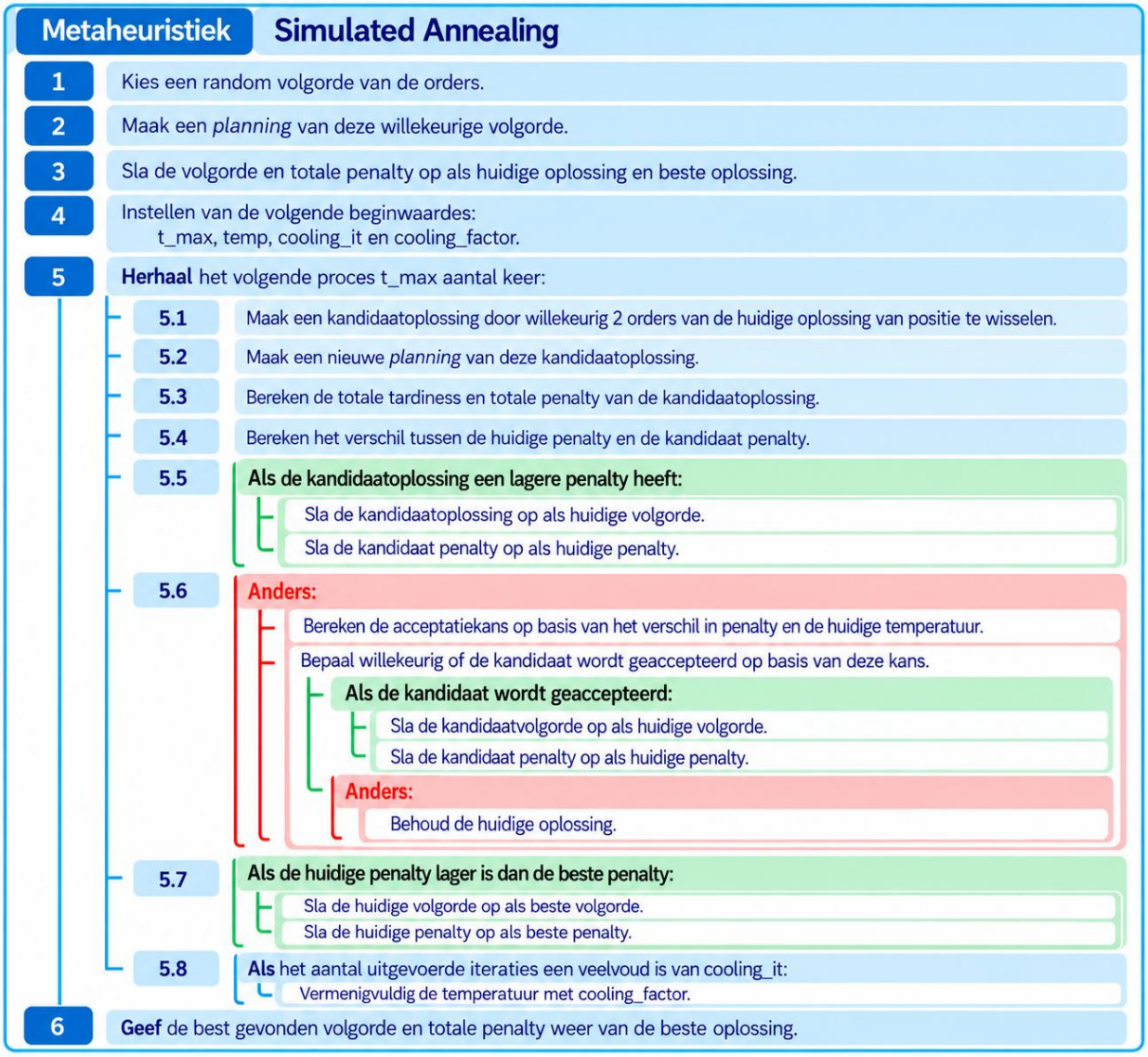
*Functie: maakt een planning*




Note: The pseudocode was developed by the authors. ChatGPT (OpenAI, 2026) was used to create the visual layout.

**Figuur 2.5**

*Simulated Annealing*




Note: The pseudocode was developed by the authors. ChatGPT (OpenAI, 2026) was used to create the visual layout.

## 2.5 Theoretische tijdscomplexiteit

**Tabel 2.1**

*Greedy constructive heuristic(EDD)*

| **Onderdeel** | **Verklaring** | **Tijdscomplexiteit** | **Binnen lus?** | **Aantal keer** | **Totaal** |
|---|---|---|---|---|---|
| Orders sorteren | Sorteren op deadline | $O(n \log n)$ | Nee | 1 | $O(n \log n)$ |
| Machines initialiseren | Lijsten aanmaken voor alle machines | $O(m)$ | Nee | 1 | $O(m)$ |
| Machine selecteren | Alle machinetijden controleren en snelste kiezen bij gelijke tijden | $O(m)$ | For-loop | $n$ | $O(m * n)$ |
| Kleuren vergelijken | 2 kleuren met elkaar vergelijken | $O(1)$ | For-loop | $n$ | $O(n)$ |
| Setuptijd zoeken | Setupregels één voor één doorzoeken | $O(s)$ | For-loop | $n$ | $O(n * s)$ |
| Productietijd berekenen | Eén deling (de breuk) | $O(1)$ | For-loop | $n$ | $O(n)$ |
| | | | | | |
| Lijsten initialiseren | Tijden en kleuren per machine aanmaken | $O(m)$ | Nee | 1 | $O(m)$ |
| Orders-dictionairy opbouwen | Orders opslaan onder ordernummer | $O(1)$ | For-loop | $n$ | $O(n)$ |
| Setuptijd zoeken | Setupregels 1 voor 1 doorzoeken | $O(s)$ | For-loop | $n$ | $O(ns)$ |
| Tijden, tardiness en kosten berekenen | Rekenstappen per order | $O(1)$ | For-loop | $n$ | $O(n)$ |
| Resultaten in volgorde zetten | Waarden uit resultaten in lijsten plaatsen | $O(1)$ | For-loop | $n$ | $O(n)$ |

$$n = aantal\ orders$$
$$m = aantal\ machines$$
$$s = aantal\ setupregels$$

De orders worden op deadline gesorteerd met de Python-functie sort(). Deze gebruikt het sorteeralgoritme Timsort, dat in het slechtste geval een tijdscomplexiteit heeft van O(n log n) (Auger et al., 2018, p. 1).

**Totale tijdscomplexiteit**

Omdat deze onderdelen na elkaar worden uitgevoerd, tellen we de complexiteiten van greedy_schedule en calculate results bij elkaar op:

$$O(n\log(n) + n + m + nm + 2 * ns + 4 * n)$$

In deze berekening worden de Excel-export en het creëren van de Gantt-chart niet meegenomen.

Bij big O wordt er gekeken naar de bewerkingen die het snelst groeien als de probleemomvang groter wordt. Hierom kunnen de losse termen $n$ en $m$ worden weglaten, omdat deze al worden meegenomen door $nm$. Voor de setup controles $ns$ is ruim gerekend, niet iedere order heeft een kleurwisseling.  Hiervoor moet wel gelden dat er minstens 1 order en 1 machine is. Hieruit volgt dat de uiteindelijke tijdscomplexiteit als volgt is:

$$O\big(n\ log(n) + n(m + s)\big)$$

**Tabel 2.2**

*Improving search (swap)*

| **Onderdeel** | **Verklaring** | **Tijdscomplexiteit** | **Binnen lus?** | **Aantal keer** | **Totaal** |
|---|---|---|---|---|---|
| Eerste planning maken en doorrekenen | Greedy_schedule + calculate_results | $O\big(n\ log(n) + n(m+s)\big)$ | nee | 1 | $O\big(n\ log(n) + n(m+s)\big)$ |
| Swap uitvoeren | Lijst kopiëren en 2 orders verwisselen | $O(n)$ | 2 x for-loop 1x while | $k * n^2$ | $O(k * n^3)$ |
| Planning maken | Greedy_schedule op de gewisselde volgorde | $O\big(n(m+s)\big)$ | 2 x for-loop 1x while | $k * n^2$ | $O\begin{pmatrix} k * n^2 * \\ (n(m+s)) \end{pmatrix}$ |
| Planning doorrekenen | calculate_results | $O\big(m + n(1+s)\big)$ | 2 x for-loop 1x while | $k * n^2$ | $O(k * n^3 * s + k * n^2 * m$ |
| Penalty opslaan en vergelijken | Penalty vergelijken en opslaan, beste volgorde kopieren | $O(n)$ | 2 x for-loop 1x while | $k * n^2$ | $O(k * n^3)$ |

$$n = aantal\ orders$$
$$m = aantal\ machines$$
$$s = aantal\ setupregels$$
$$k = aantal\ rondes\ van\ while - loop$$

**Totale tijdscomplexiteit**

De totale tijdscomplexiteit voor de improving search inclusief alle berekeningen is (vereenvoudiging zie uitleg pagina 18):

$$O(k * n^3 * (m+s)$$

Hieruit is te zien dat het aantal orders (n) de grootste invloed heeft op de rekentijd. Het aantal rondes werkt lineair door op de rekentijd (Srinivasan & Lundqvist, 2005). Ook het aantal machines (m) en setupregels (s) beïnvloedt de rekentijd, afhankelijk van de grootte van deze variabelen.

**Tabel 2.3**

Meta heuristiek Simulated Annealing (SA)

| **Onderdeel** | **Verklaring** | **Tijdscomplexiteit** | **Binnen lus?** | **Aantal keer** | **Totaal** |
|---|---|---|---|---|---|
| Beginoplossing maken | Lijst maken en random.shuffle | $O(n)$ | Nee | 1 | $O(n)$ |
| Beginoplossing doorreken | calculate_tard_pen op de beginvolgorde | $O(n(m+s))$ | Nee | 1 | $O(n(m+s))$ |
| Random swap | 2 orders van positie swappen | $O(n)$ | For-loop | t_max | $O(n * t_max)$ |
| Nieuwe oplossing doorreken | calculate_tard_pen op de nieuwe volgorde | $O(n(m+s))$ | For-loop | t_max | $O(t_max * n(m+s))$ |
| Acceptatiekans bepalen | m.exp, random.choices en vergelijking | $O(1)$ | For-loop | t_max | $O(t_max)$ |
| Oplossing accepteren/ Beste bewaren | Current en best kopieren | $O(n)$ | For-loop | t_max | $O(n * t_max)$ |
| Temperatuur verlagen | temp*cooling_factor | $O(1)$ | For-loop | t_max | $O(t_max)$ |

$$n = aantal\ orders$$
$$m = aantal\ machines$$
$$s = aantal\ setupregels$$
$$t_\max = maximaal\ aantal\ iteraties$$

**Totale tijdscomplexiteit**

De totale tijdscomplexiteit voor de SA inclusief het berekenen en doorreken van de oplossingen is:

$$O(t_{max} * n * (m+s))$$

Hierin is te zien dat de rekentijd van SA afhangt van het aantal orders (n) en het aantal iteraties (t_max). Ook het aantal machines (m) en het aantal setupregels (s) tellen mee, want deze staan in dezelfde factor als n. Bij improving search worden per ronde $n^2$ swaps getest waardoor één ronde veel meer rekentijd kost dan één iteratie van SA.

# 3 Implementatie van de oplosmethoden

In dit hoofdstuk beschrijven we hoe de ontworpen oplossingsmethoden in Python zijn gerealiseerd. Wij maken duidelijk hoe de implementatie aansluit op het ontwerp uit hoofdstuk 2 en hoe ervoor is gezorgd dat het programma bruikbaar is voor verschillende probleeminstanties

## 3.1 Opbouw van het Python-programma

Ons Python-programma is opgedeeld in meerdere bestanden, waarbij iedere oplossingsmethode (Greedy, Improving Search en Simulated Annealing) in een afzonderlijk bestand is geïmplementeerd.

In Functions.py staan algemene functies die door meerdere oplossingsmethoden worden gebruikt. Zo worden gantt_chart en gantt_chart_list gebruikt om de gemaakte planningen te visualiseren in een Gantt-chart. De functies resultaten_naar_excel en resultaten_naar_excel_list worden gebruikt om de resultaten van een planning te exporteren naar een Excelbestand. Daarnaast staan in dit bestand de functies swap en random_swap, waarmee twee orders in een volgorde kunnen worden verwisseld. Wij hebben ervoor gekozen om deze algemene functies in een apart bestand te plaatsen, zodat dezelfde functies vanuit meerdere bestanden kunnen worden gebruikt en de code overzichtelijk blijft.

De eerste oplossingsmethode begint in Greedy.py. Hier worden de gegevens uit het PaintShop-Excelbestand ingelezen. Vervolgens wordt met greedy_schedule een initiële planning opgebouwd. De orders worden hierbij op deadline gesorteerd en toegewezen aan de machine met de laagste huidige machinetijd. Wanneer meerdere machines dezelfde huidige tijd hebben, wordt de snelste machine gekozen. Met calculate_results worden vervolgens onder andere de setup-tijden, procestijden, eindtijden, tardiness en penaltykosten van de planning berekend.

De Greedy-planning wordt vervolgens in Improved_greedy.py verder verbeterd met Discrete Improving Search. Hierbij worden verschillende ordervolgordes onderzocht door twee orders met elkaar te verwisselen. Alle mogelijke swaps worden onderzocht en de beste verbeterende swap wordt gekozen. Dit proces wordt herhaald totdat geen enkele swap meer leidt tot een lagere totale penalty.

De tweede oplossingsmethode wordt uitgevoerd in Metaheuristic.py. Hier wordt Simulated Annealing toegepast. Eerst wordt een willekeurige beginvolgorde van de orders gemaakt. Vervolgens worden met random_swap steeds twee willekeurige orders verwisseld om een nieuwe kandidaatoplossing te maken. Een oplossing met een lagere penalty wordt geaccepteerd. Een slechtere oplossing kan met een bepaalde kans ook worden geaccepteerd. Deze kans is afhankelijk van de temperatuur. Tijdens het algoritme wordt de temperatuur volgens de ingestelde cooling_factor en cooling_it geleidelijk verlaagd.

De beste oplossing van Simulated Annealing wordt in Improved_metaheuristic.py verder verbeterd met Discrete Improving Search. Hierbij worden systematisch alle mogelijke swaps tussen twee orders onderzocht. Voor iedere swap wordt de totale penalty berekend. Met Best Improvement wordt de swap gekozen die de grootste verbetering oplevert. Dit proces wordt herhaald totdat geen enkele swap meer een lagere totale penalty oplevert. Op dat moment is een lokaal optimum binnen de gekozen swap-neighbourhood bereikt.

De verschillende PaintShop-Excelbestanden vormen de invoer van het programma en bevatten de gegevens over de orders, machines en setup-tijden. De gegenereerde planningen worden met resultaten_naar_excel of resultaten_naar_excel_list geëxporteerd naar de Results_...-Excelbestanden. Daarnaast worden de uiteindelijke planningen weergegeven in een Gantt-chart. Voor de verbeterde metaheuristische methode wordt met plot_penalty ook het verloop van de penalty tijdens de Improving Search gevisualiseerd.

Tot slot hebben we een aparte map aangemaakt met testsets. Met deze datasets kunnen we verschillende onderdelen van de code testen volgens het door ons opgestelde testplan. De testsets zijn opgeslagen in de map Data_test_sets.

## 3.2 Overeenkomst tussen ontwerp en implementatie

De implementatie van ons Python-programma is gebaseerd op de algoritmen en ontwerpkeuzes die in hoofdstuk 2 zijn beschreven. Hierbij hebben we gebruikgemaakt van de Greedy Constructive Heuristic, Discrete Improving Search en Simulated Annealing.

**Greedy Constructive Heuristic**

Zoals beschreven in paragraaf 2.2, hebben we gekozen voor de EDD-regel (Earliest Due Date). Deze is geïmplementeerd in Greedy.py, in de functie greedy_schedule(). Hierbij worden de orders eerst gesorteerd op deadline en vervolgens één voor één toegewezen aan de machine met de laagste huidige machinetijd. Wanneer meerdere machines dezelfde machinetijd hebben, wordt de snelste machine gekozen. Ook wordt rekening gehouden met eventuele kleurwisselingen en de bijbehorende setup-tijden. Met de functie calculate_results() worden vervolgens onder andere de productietijd, tardiness en totale penalty berekend.

**Discrete Improving Search**

In paragraaf 2.3 hebben we gekozen voor de swap-methode als buurruimtestructuur en Best Improvement als zoekstrategie. Deze methode is verwerkt in Improved_greedy.py. Hierbij worden telkens twee orders van positie verwisseld en wordt de totale penalty van de nieuwe planning berekend. Volgens het ontwerp worden alle mogelijke swaps onderzocht en wordt de oplossing met de grootste verbetering gekozen. Dit proces wordt herhaald

totdat geen enkele swap meer een lagere penalty oplevert. De stopconditie is daarmee gebaseerd op het bereiken van een lokaal optimum binnen de gekozen swap-buurruimte.

**Simulated Annealing**

De metaheuristiek uit paragraaf 2.4 is geïmplementeerd in Metaheuristic.py, in de functie SA(). Het algoritme begint met een willekeurige ordervolgorde en genereert nieuwe oplossingen door telkens twee willekeurige orders te verwisselen. Een oplossing met een lagere penalty wordt altijd geaccepteerd, terwijl een slechtere oplossing met een bepaalde kans kan worden geaccepteerd. Deze kans is afhankelijk van het verschil in penalty en de huidige temperatuur.

Voor Simulated Annealing hebben we vier parameters ingesteld: t_max, temp, cooling_factor en cooling_it. In Metaheuristic.py gebruiken we 10.000 iteraties, een begintemperatuur van 1.000, een cooling_factor van 0,99 en een temperatuurverlaging na iedere 1.000 iteraties. Zodra het maximale aantal iteraties is bereikt, stopt het algoritme en wordt de beste gevonden oplossing teruggegeven.

In Improved_metaheuristic.py wordt de beste oplossing van Simulated Annealing verder verbeterd met Discrete Improving Search. Hierbij worden systematisch alle mogelijke swaps onderzocht en wordt telkens de beste verbetering gekozen. Dit proces wordt herhaald totdat geen enkele swap meer een lagere totale penalty oplevert.

Tijdens de experimenten zijn deze parameters aangepast om de oplossingskwaliteit te verbeteren. Voor de standaarduitvoering zijn lagere instellingen gebruikt om de rekentijd te beperken.

## 3.3 Invoer, uitvoer en generieke toepasbaarheid

De probleeminstanties worden ingelezen vanuit een Excelbestand met behulp van pd.read_excel. Het Excelbestand bevat de tabbladen Orders, Machines en Setups. Deze worden afzonderlijk ingelezen en vervolgens met de functie dictionary omgezet naar lijsten met dictionaries, zodat de gegevens verder in de oplossingsmethoden kunnen worden gebruikt. Op basis van deze gegevens wordt een planning gemaakt en worden onder andere de gekozen machine, het volgnummer, de setup-tijd, begin- en eindtijd, procestijd, tardiness en penalty per order berekend. De uiteindelijke planning wordt met resultaten_naar_excel of resultaten_naar_excel_list geëxporteerd naar een Excelbestand. Daarnaast kan de planning met een Gantt-chart worden gevisualiseerd.

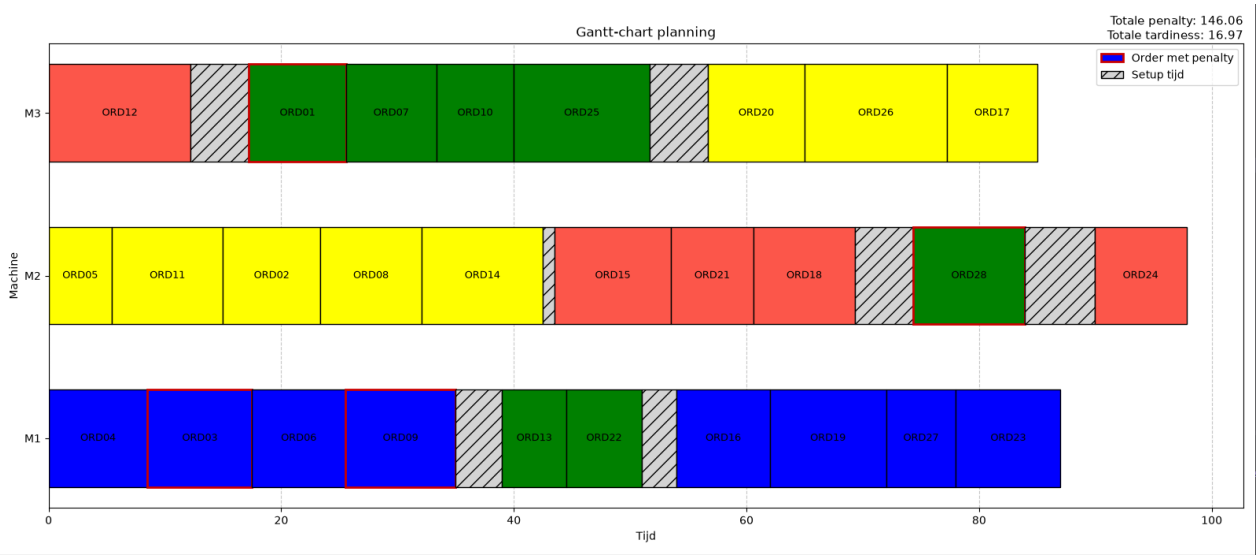
De oplossingsmethoden zijn zo opgebouwd dat ze ook kunnen worden toegepast op andere probleeminstanties met een ander aantal orders en machines, zolang het invoerbestand dezelfde structuur en kolomnamen heeft. De aantallen orders en machines

zijn namelijk niet vast in de algoritmen opgenomen, maar worden bepaald aan de hand van de ingelezen gegevens. Wanneer een andere probleeminstantie wordt gebruikt, moet op dit moment alleen de bestandsnaam bij pd.read_excel(...) worden aangepast naar het gewenste PaintShop-Excelbestand. De logica van de greedy heuristic, Improving Search en Simulated Annealing hoeft hiervoor niet te worden aangepast.

## 3.4 Visualisatie van planningen

**Figuur 3.1**

*Voorbeeld van gegenereerde planning weergegeven in een Gantt-chart*




De gegenereerde planningen worden gevisualiseerd met een Gantt-chart. Hierin wordt per machine weergegeven welke orders worden uitgevoerd en op welke tijdstippen deze worden verwerkt. De kleur van iedere order komt overeen met de bijbehorende verfkleur. De gearceerde vlakken geven de benodigde omsteltijd (setup-tijd) tussen verschillende kleuren weer. Orders die na hun deadline worden afgerond, krijgen een rode rand. Rechtsboven in de Gantt-chart worden de totale tardiness en totale penalty van de planning weergegeven. De Gantt-chart maakt het mogelijk om de verdeling en volgorde van de orders over de machines overzichtelijk te bekijken. Daarnaast kan visueel worden gecontroleerd waar omsteltijden optreden en welke orders te laat worden afgerond.

## 3.5 Leesbaarheid en documentatie van de code

Om de code leesbaar en onderhoudbaar te houden, hebben we het programma logisch opgedeeld in verschillende bestanden en functies. De belangrijkste onderdelen van de oplossingsmethoden zijn ondergebracht in afzonderlijke functies met beschrijvende namen, zoals greedy_schedule, calculate_results, random_swap en resultaten_naar_excel. Voor variabelen zijn zoveel mogelijk namen gebruikt die aangeven welke gegevens ze bevatten, zoals setup_per_order, tardiness_per_order en machines_per_order. Bij de belangrijkste functies is met docstrings kort beschreven wat de functie doet en welke resultaten worden teruggegeven. Daarnaast is commentaar bij belangrijke stappen in de algoritmen, bijvoorbeeld bij het berekenen van de setup- en procestijd, het kiezen van een machine en het accepteren van oplossingen bij Simulated Annealing. Algemene functies

die door meerdere oplossingsmethoden worden gebruikt, zijn samengevoegd in Functions.py, zodat dubbele code zoveel mogelijk wordt voorkomen en de verschillende Python-bestanden overzichtelijk blijven.

# 4 Validatie

In dit hoofdstuk controleren we of de implementatie correct werkt en of de gegenereerde planningen aan de gestelde voorwaarden voldoen. Hiervoor gebruiken we een testplan met passende testgegevens en controleerbare resultaten.

## 4.1 Testopzet

Om te controleren of ons Python-programma correct werkt, hebben we een testplan opgesteld met in totaal 13 tests. Hiervoor hebben we verschillende Excel-testsets aangemaakt, die zijn opgeslagen in de map Data_test_sets. Voor het uitvoeren van de tests hebben we aparte Python-bestanden gebruikt, namelijk Improved_greedyForTest.py en Metaheuristic_ForTest.py. Hierdoor konden we verschillende onderdelen van onze oplossingsmethoden testen zonder de oorspronkelijke code aan te passen.

De tests zijn onderverdeeld in drie categorieën: Happy Flow, Error Cases en Boundary Cases. Bij de Happy Flow-tests controleren we onder andere de berekeningen van de productietijd, tardiness en penalty, de machineselectie, de setup-tijden en de werking van de swap-methode. Bij de Error Cases controleren we of het programma ontbrekende of ongeldige invoergegevens herkent. Tot slot onderzoeken we bij de Boundary Cases hoe het programma omgaat met orders die precies op of net na hun deadline eindigen.

Voor iedere test hebben we vooraf het verwachte resultaat bepaald. Dit hebben we gedaan door berekeningen handmatig uit te voeren of door vast te stellen hoe het programma op een bepaalde situatie moet reageren. Vervolgens hebben we de resultaten van Python vergeleken met deze verwachte uitkomsten. Wanneer de resultaten overeenkomen, wordt de test beoordeeld met PASS. Bij een afwijking wordt de test beoordeeld met FAIL.

Het volledige testplan, inclusief de testgegevens, verwachte resultaten en werkelijke uitkomsten, is opgenomen als apart Excelbestand in ons portfolio.

## 4.2 Uitgevoerde tests en resultaten

De 13 tests uit ons testplan zijn uitgevoerd met de verschillende testsets. Hierbij hebben we de werkelijke resultaten van het Python-programma vergeleken met de vooraf bepaalde verwachte resultaten. Zoals te zien is in **figuur 4.1**, hebben alle 13 tests volgens ons testoverzicht het verwachte resultaat opgeleverd en zijn deze beoordeeld met PASS. Er zijn daarom geen aanvullende acties nodig.

Als extra controle hebben we bij Test 01 (Happy Flow) een aantal berekeningen handmatig uitgevoerd. Bij de eerste handmatige controle kwamen de berekende resultaten niet overeen met de uitkomsten van de python-code. Na onderzoek bleek dat de machines ook

op verwerkingssnelheid werden gesorteerd, waardoor bij de verdere berekeningen een verkeerde machinevolgorde werd gebruikt. Nadat we hiermee rekening hadden gehouden, kwamen de berekeningen wel overeen met de resultaten uit de python-code.

Daarnaast hebben we voor ORDER02 de productietijd, eindtijd en tardiness berekend. De productietijd bedraagt 8 tijdseenheden (200 / 25). Met een begintijd van 17,2 resulteert dit in een eindtijd van 25,2. Omdat de deadline 28 is, bedraagt de tardiness 0. Deze resultaten komen overeen met de uitkomsten van de Python-code. De handmatige berekeningen en de vergelijking met de resultaten van Python zijn weergegeven in **figuur 4.2**.

Naast deze handmatige controle hebben we onder andere getest of het programma ontbrekende invoergegevens herkent, of er geen setup-tijd wordt toegevoegd bij opeenvolgende orders met dezelfde kleur en of de penalty correct wordt berekend wanneer een order te laat wordt afgerond. De resultaten van deze tests zijn eveneens weergegeven in **figuur 4.1**. Het volledige testplan is daarnaast als apart Excelbestand opgenomen in ons portfolio.

**Figuur 4.1**

*Overzicht van uitgevoerde tests en bijbehorende resultaten*

| Test ID | Type | What is to be validated | Test data | Expected Result | Result | Conclusion | Follow up action |
|---|---|---|---|---|---|---|---|
| 1 | Happy flow | Controleren of productietijd, tardiness en penalty correct worden berekent | Test01_HappyFlow | Resultaten zijn gelijk aan de handmatige berekening | As expected | PASS | None |
| 2 | Happy flow | Controleren of bij gelijke machinetijd de snelste machine wordt gekozen | Test08_EqualMachineTime | Order wordt toegewezen aan de snelste machine | As expected | PASS | None |
| 3 | Happy flow | Controleren of een order wordt toegewezen aan de machine met de laagste huidige tijd | Test01_HappyFlow | Order wordt toegewezen aan de machine met de laagste eindtijd | As expected | PASS | None |
| 4 | Happy flow | Controleren of de penalty per order correct wordt berekend | Test09_Penalty | Totale penalty voor de order = 50 | As expected | PASS | None |
| 5 | Happy flow | Controleren of swap 2 orders verwisseld | Test01_HappyFlow | Nieuwe volgorde [ORD03, ORD02, ORD01] | As expected | PASS | None |
| 6 | Happy flow | Controleren of de penalty van de beste SA-oplossing overeenkomt met een nieuwe berekening | Test01_HappyFlow | opnieuw berekende penalty = best_penalty | As expected | PASS | None |
| 7 | Happy flow | Controleren dat geen setup-tijd wordt toegevoegd wanneer twee opeenvolgende orders dezelfde kleur hebben | Test05_SameColour | Geen setup-tijd tussen de twee orders | As expected | PASS | None |
| 8 | Happy flow | Controleren of bij een kleurwisseling de juiste setup-tijd wordt toegevoegd | Test01_HappyFlow | Setup tijd van Red -> Blue wordt toegevoegd | As expected | PASS | None |
| | | | | | | | |
| 9 | Error case | Controleren of het programma een ontbrekende kolom 'deadline' herkent | Test02_MissingDeadline | Foutmelding dat de kolom 'deadline' ontbreekt | As expected | PASS | None |
| 10 | Error case | Controleren of een machinesnelheid van 0 correct wordt afgehandeld | Test03_SpeedZero | Foutmelding dat de machinesnelheid ongeldig is | As expected | PASS | None |
| 11 | Error case | Controleren of een ontbrekende setup wordt herkent | Test04_MissingSetup | Foutmelding: geen setup gevonden van Blue naar Red | As expected | PASS | None |
| | | | | | | | |
| 12 | Boundary case | Controleren wat er gebeurt als een order exact op de deadline eindigt | Test06_ExactDeadline | Tardiness = 0 en totale penalty voor de order = 0 | As expected | PASS | None |
| 13 | Boundary case | Controleren wat er gebeurt als een order 1 tijdseenheid na de deadline eindigt | Test07_JustAfterDeadline | Tardiness = 1 en penalty = 1 * penaltyfactor | As expected | PASS | None |

**Parameterlijst:**

$i = 1 \ldots n; hierbij\ is\ i\ het\ ordernummer\ en\ n\ het\ totale\ aantal\ orders$
$m = \ 1 \ldots n;\ hierbij\ is\ m\ het\ machinenummer\ en\ n\ het\ totale\ aantal\ machines$

$T_i = tardiness\ van\ order\ i$
$Di = deadline\ van\ order\ i$
$P_i = totale\ penalty\ order\ i$
$p_i = penaltyfactor\ order\ i\ (per\ tijdseenheid)$
$S_i = starttijd\ order\ i$
$E_i = eindtijd\ order\ i$
$M_i = productietijd\ order\ i$
$o_i = opp\ order\ i$
$s_m = speed\ machine\ m$
$M_i = \frac{o_i}{s_m}$

**Figuur 4.2**

*Handmatige controle van de productietijd, eindtijd en tardiness voor Test01*

$$T_i = \max(0, E_i - D_i)$$

$$E_i = S_i + \ M_i$$

$$P_i = T_i \times p_i$$

$We\ voeren\ de\ test\ uit\ voor\ ORDER02\ (i = 2)$:

$$M_i = \frac{200}{25} = 8$$

$$T_2 = \max(0, 25.2 - 28) = 0$$

$$E_i = (7.2 + 10) + 8$$

$$P_2 = \ 0 \ \times 12 = 0$$

## 4.3 Validatie van lokale optimaliteit

Om te controleren of onze Discrete Improving Search eindigt in een lokaal optimum, onderzoeken we alle mogelijke swaps tussen twee orders. Voor iedere swap wordt de planning opnieuw berekend en wordt de totale penalty vergeleken met de beste gevonden penalty. Wanneer een swap een lagere penalty oplevert, wordt de beste verbetering gekozen en wordt het proces opnieuw uitgevoerd. Dit wordt herhaald totdat geen enkele swap meer een lagere penalty oplevert.

In Improved_greedy.py wordt dit gecontroleerd met de while verbetering-loop. Wanneer na het onderzoeken van alle mogelijke swaps geen verbetering meer wordt gevonden, stopt het algoritme. Hiermee wordt een lokaal optimum bereikt binnen de gekozen swap-buurruimte. Dit betekent niet dat de gevonden planning ook het globale optimum is.

## 4.4 Conclusie van de validatie

Uit de uitgevoerde tests blijkt dat ons Python-programma op de geteste onderdelen de verwachte resultaten oplevert. Alle 13 tests zijn volgens ons testoverzicht geslaagd. Daarnaast hebben we met handmatige berekeningen gecontroleerd of de productietijd, eindtijd en tardiness correct worden berekend. Deze resultaten kwamen overeen met de uitkomsten van Python.

Ook hebben we gecontroleerd of het programma correct omgaat met verschillende situaties, zoals ontbrekende invoergegevens, kleurwisselingen en orders die precies op of na hun deadline eindigen. Tijdens de uitgevoerde tests zijn volgens ons testoverzicht geen afwijkingen gevonden die aanvullende acties vereisten. Hiermee hebben we aangetoond dat de geteste onderdelen van onze implementatie correct functioneren.

# 5 Experimenten

In dit hoofdstuk bespreken we het onderzoek naar de prestatie van de oplosmethoden aan de hand van rekenexperimenten. Wij gebruiken de resultaten van experimenten om onderbouwde keuzes te maken voor de uiteindelijke oplossingsmethode en geschikte parameterwaarden

## 5.1 Experimentele opzet

Met onze experimenten onderzoeken we welke invloed verschillende parameterinstellingen hebben op de prestaties van Simulated Annealing. Hierbij willen we antwoord geven op de volgende vragen:

- Wat is de invloed van het maximale aantal iteraties (t_max) op de totale penalty en rekentijd?
- Welke cooling factor levert de beste resultaten op?
- Wat is het effect van het aantal iteraties tussen temperatuurverlagingen (cooling_it) en de begintemperatuur (temp)?

Daarnaast onderzoeken we in hoeverre verschillende random seeds invloed hebben op de gevonden oplossingen.

Voor de experimenten gebruiken we de PaintShop-dataset van september 2026. We beginnen met vijf experimentenreeksen waarin we verschillende parameterinstellingen onderzoeken. Als standaardinstellingen gebruiken we:

- t_max = 500.000
- cooling_factor = 0,99
- cooling_it = 1000
- temp = 1000
- random seed = 1

Het parameteronderzoek is in meerdere fasen uitgevoerd. Op basis van de eerste experimentenreeksen zijn aanvullende experimenten uitgevoerd met meerdere random seeds en combinaties van parameterinstellingen. In de zes bijbehorende septemberbestanden zijn in totaal 445 runs geregistreerd. Daarnaast zijn voor de novemberdataset 108 runs uitgevoerd om geschikte parameterinstellingen voor deze grotere probleeminstantie te onderzoeken.

Iedere run bestaat uit één uitvoering van Simulated Annealing, gevolgd door Discrete Improving Search. Hierbij registreren we de totale penalty, totale tardiness en rekentijd vóór en na Improving Search. Hiermee onderzoeken we welke parameterinstellingen een goede

balans opleveren tussen oplossingskwaliteit, consistentie en rekentijd. Ook bepalen we hoeveel extra verbetering Discrete Improving Search oplevert ten opzichte van Simulated Annealing alleen. De resultaten worden in de volgende paragrafen besproken.

## 5.2 Onderzoek naar parameterinstellingen

Er is gekozen om de experimenten uit te voeren met de PaintShop-dataset van september. Deze dataset is relatief klein, waardoor de benodigde rekentijd beperkt blijft en de experimenten sneller kunnen worden uitgevoerd. De rekentijd van Simulated Annealing neemt ongeveer lineair toe met het maximale aantal iteraties t_max. Dit betekent dat wanneer het aantal iteraties tien keer zo groot wordt, de rekentijd naar verwachting ook ongeveer tien keer zo lang wordt. Dit verband wordt verder toegelicht in paragraaf 2.5.

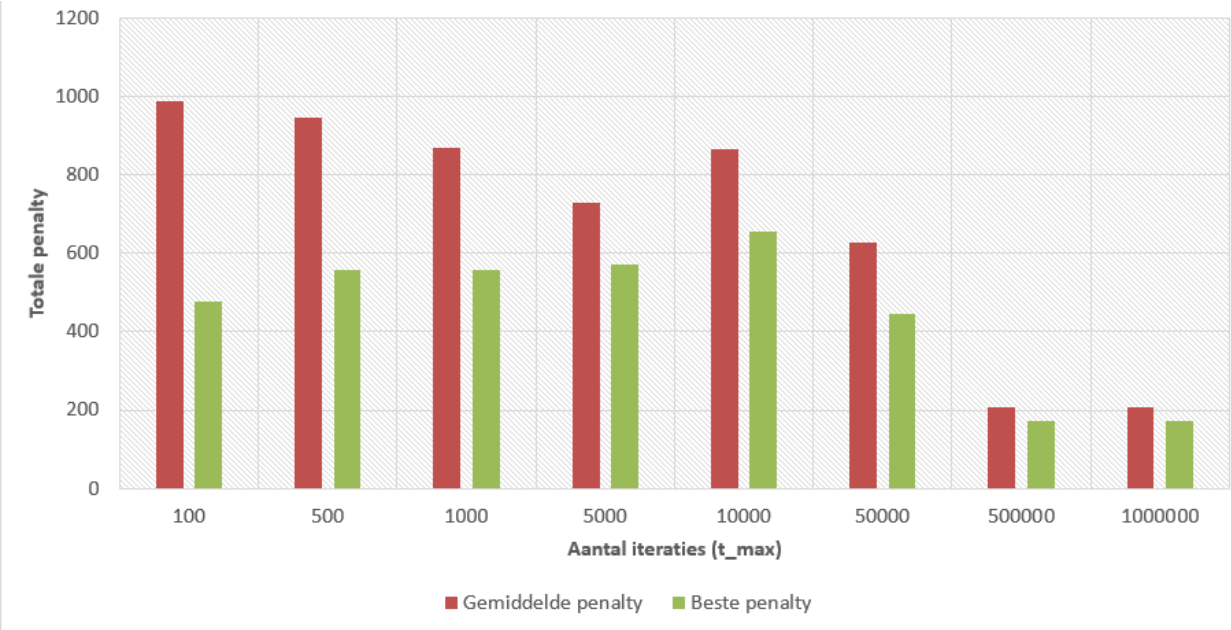
Bij het selecteren van de parameterinstellingen kijken we niet alleen naar de laagste gevonden penalty, maar ook naar de gemiddelde totale penalty en de standaarddeviatie over meerdere runs. Tijdens de experimenten zijn met andere parameterinstellingen lagere afzonderlijke penalties gevonden. We kiezen echter voor instellingen die gemiddeld goede resultaten opleveren met een relatief kleine spreiding. Hiermee streven we naar een goede balans tussen oplossingskwaliteit en consistentie.

In de volgende experimenten onderzoeken we de invloed van het aantal iteraties, de begintemperatuur en het cooling interval op de totale penalty.

**Experiment 1:** Instellen van het totale aantal iteraties

**Figuur 5.1**

*Invloed van het aantal iteraties op de gemiddelde een beste totale penalty*




Om het geschikte aantal iteraties te bepalen, hebben we acht verschillende aantallen iteraties getest. Iedere instelling is vijf keer uitgevoerd met verschillende random seeds. Vervolgens hebben we per instelling de gemiddelde totale penalty berekend.

In **Figuur 5.1** is te zien dat de gemiddelde totale penalty vanaf ongeveer 500.000 iteraties nauwelijks meer afneemt. Om voldoende ruimte te houden voor eventuele verdere verbeteringen, kiezen we voor 700.000 iteraties. Omdat een hoger aantal iteraties ook leidt tot een langere rekentijd, beschouwen we 700.000 iteraties als een goede balans tussen de kwaliteit van de oplossing en de benodigde rekentijd.

**Experiment 2**: Invloed van de temperatuurinstellingen

Experiment 2.1: Analyse van de temperatuurinstellingen

Om de juiste verhoudingen te bepalen van het aantal cooling-iteraties, de cooling factor en de temperatuur wordt er weer een aantal sets gedraaid waarvan het gemiddelde wordt genomen. Uiteindelijk willen we dat na het t_max aantal iteraties er nog 1 procent van de temperatuur overblijft.

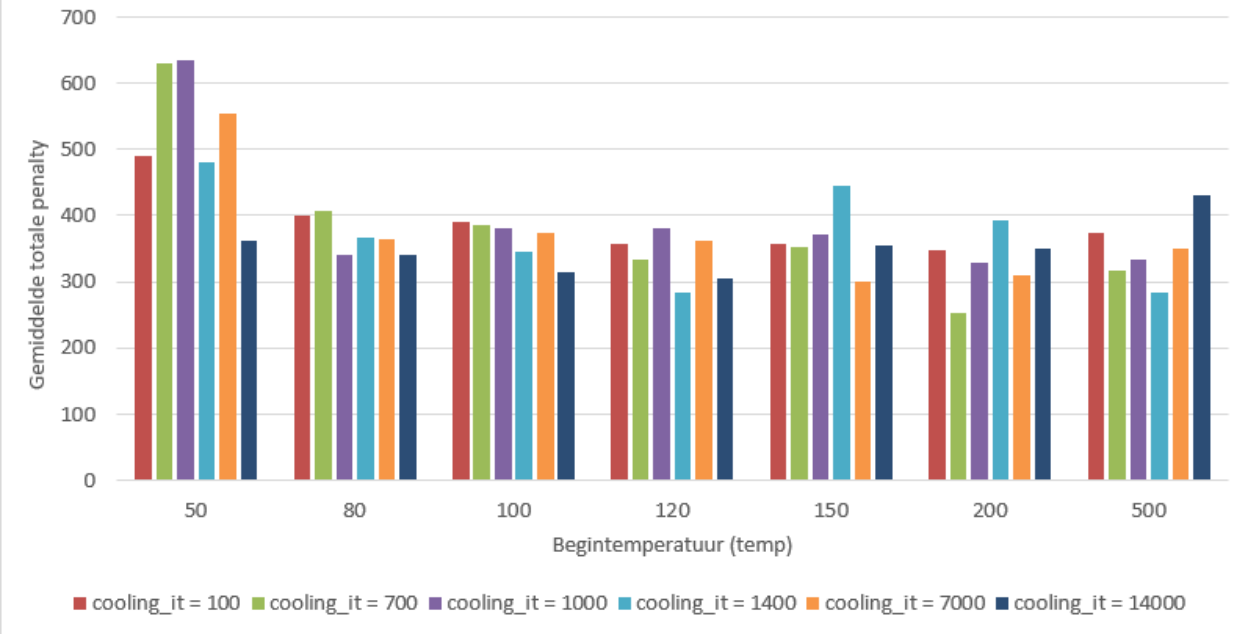
De cooling factor wordt dan:

$$Cooling_factor = 0.01^{\frac{1}{k}}$$
$$k = \frac{t_max}{cooling_it}$$

Om de rekentijd te beperken, is ervoor gekozen om het parameteronderzoek uit te voeren met 70.000 iteraties. Hierbij wordt rekening gehouden met de balans tussen de benodigde rekentijd en de kwaliteit van de gevonden oplossingen. Door minder iteraties te gebruiken, kunnen meer parametercombinaties worden getest binnen een beperkte tijd.

**Figuur 5.2**

*Invloed van begintemperatuur en cooling-interval op de gemiddelde penalty*

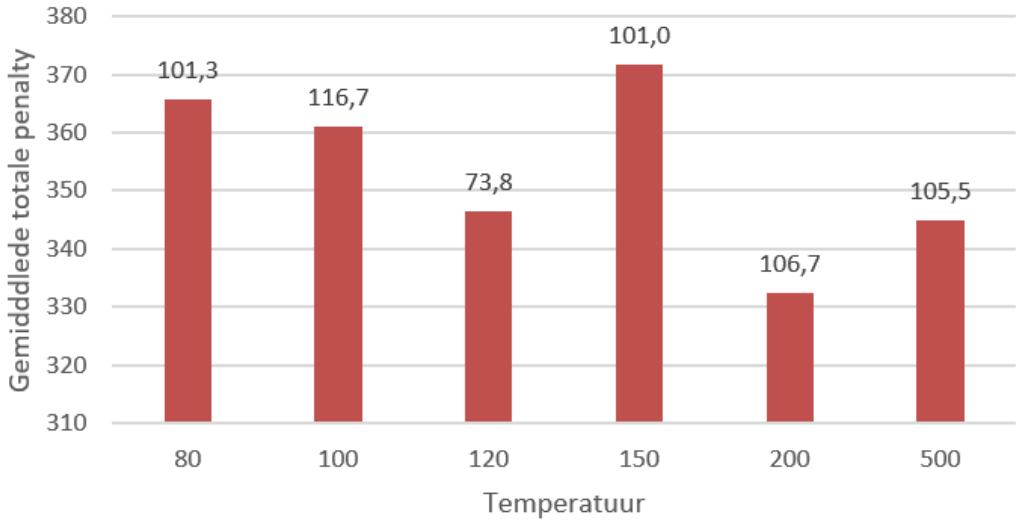



In **figuur 5.2** is te zien dat een begintemperatuur van 50 significant slechtere oplossingen geeft met een hogere gemiddelde totale penalty. Onze vervolgstap hierop is om alle gegevens van de temperatuur van 50 te verwijderen.

Experiment 2.2: Selectie van geschikte begintemperaturen

**Figuur 5.3**

*Gemiddelde totale penalty per begintemperatuur. De getallen boven de staven geven de standaarddeviatie weer.*




In **figuur 5.3** is te zien dat de gemiddelde totale penalty bij een begintemperatuur van 80 en 100 relatief hoog is ten opzichte van de overige temperaturen. Daarom besluiten we deze twee temperaturen uit te sluiten van het verdere onderzoek.

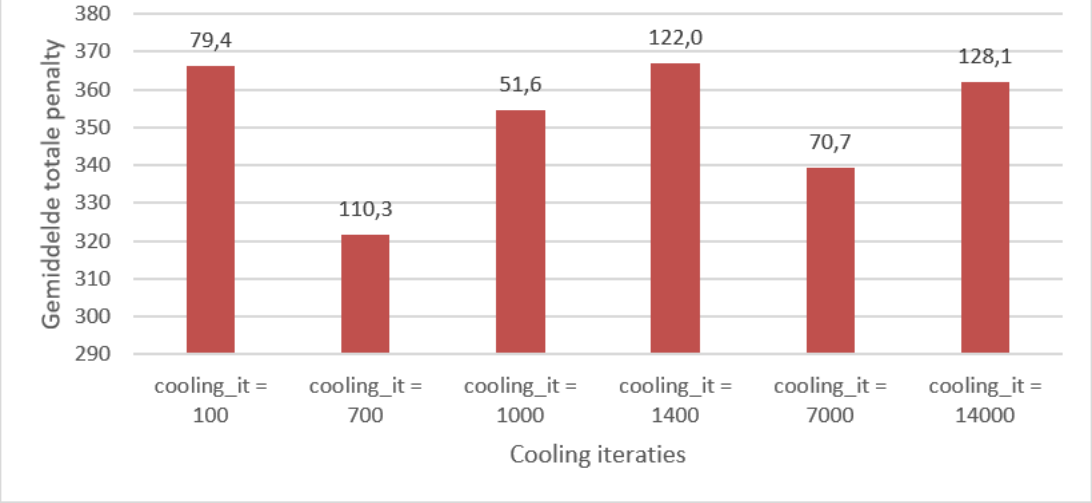
Bij een begintemperatuur van 150 is een afwijkende stijging van de gemiddelde penalty zichtbaar. De standaarddeviatie wijkt echter niet opvallend af van die van de andere temperaturen. Hierdoor kunnen we geen duidelijke oorzaak aanwijzen voor deze stijging.

We behouden daarom de overige temperaturen en berekenen opnieuw de gemiddelde penalties om te onderzoeken welke begintemperatuur het meest geschikt is.

Experiment 2.3: Selectie van geschikte cooling-iteraties

**Figuur 5.4**

*Gemiddelde totale penalty per cooling-iteraties. De getallen boven de staven geven de standaarddeviatie weer.*



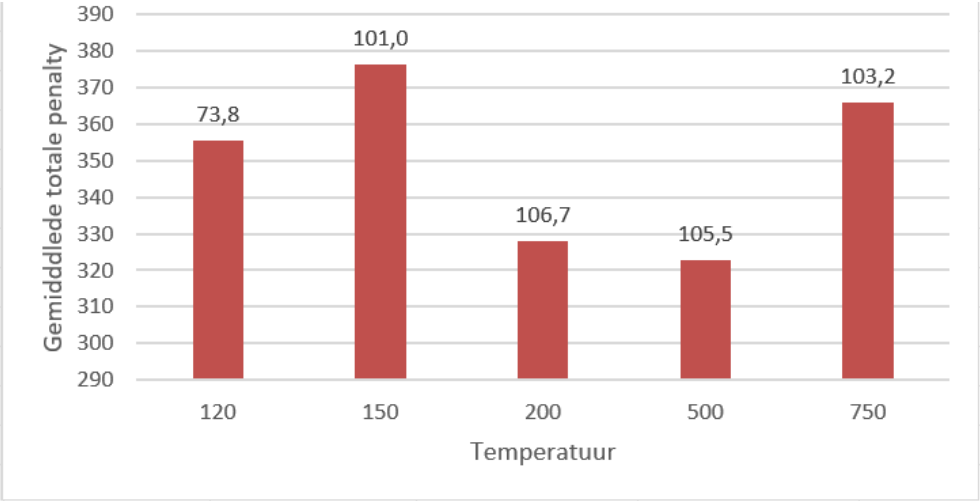

In **figuur 5.4** is te zien dat een temperatuurverlaging na iedere 700 iteraties de laagste gemiddelde totale penalty oplevert. Ook bij een temperatuurverlaging na iedere 1000 en 7000 iteraties worden relatief lage gemiddelde penalties behaald.

Bij een temperatuurverlaging na iedere 100, 1400 en 14000 iteraties ligt de gemiddelde penalty hoger. Daarnaast is de standaarddeviatie bij 1400 en 14000 iteraties relatief hoog, wat wijst op een grotere spreiding in de resultaten.

We besluiten daarom de instellingen van 100 en 14000 iteraties uit te sluiten van het verdere onderzoek. De instellingen van 700, 1000, 1400 en 7000 iteraties per temperatuurverlaging worden verder onderzocht. Hoewel 1400 iteraties een relatief hoge gemiddelde penalty en standaarddeviatie heeft, behouden we deze instelling omdat deze tussen twee instellingen ligt die wel gunstige resultaten opleveren, namelijk 1000 en 7000 iteraties.

Experiment 2.4: Selectie van de begintemperatuur

**Figuur 5.5** *Gemiddelde totale penalty per temperatuur. De getallen boven de staven geven de standaarddeviatie weer.*

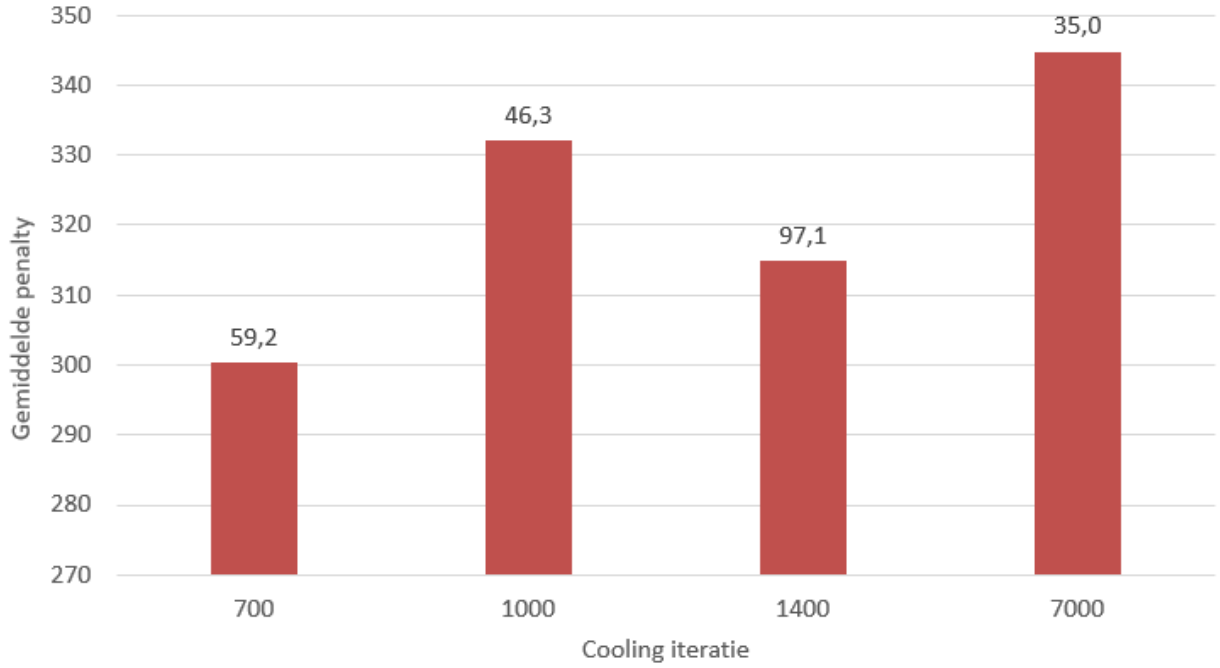



In **figuur 5.5** is te zien dat een begintemperatuur van 500 een relatief lage gemiddelde totale penalty oplevert. Daarnaast is de standaarddeviatie vergelijkbaar met die van de andere temperaturen, wat betekent dat de spreiding van de resultaten niet opvallend groot is. Daarom kiezen we een begintemperatuur van 500 voor het verdere onderzoek.

Experiment 2.5: Selectie van het aantal cooling-iteraties

**Figuur 5.6**

*Gemiddelde totale penalty per cooling-iteraties. De getallen boven de staven geven de standaarddeviatie weer.*




In **figuur 5.6** is te zien dat een temperatuurverlaging na iedere 700 iteraties de laagste gemiddelde totale penalty oplevert. Daarnaast is de standaarddeviatie van 59,2 relatief laag in vergelijking met de meeste andere instellingen. Dit wijst op een redelijk consistente prestatie.

Daarom kiezen we ervoor om de temperatuur na iedere 700 iteraties te verlagen, omdat deze instelling de beste combinatie biedt van een lage gemiddelde penalty en een acceptabele standaarddeviatie.

Tijdens het parameteronderzoek is een laagste afzonderlijke penalty van 146,06 gevonden bij 70.000 iteraties, een begintemperatuur van 100, een cooling interval van 100, een cooling factor van 0,99344 en random seed 14. Ondanks dit resultaat hebben we deze parametercombinatie niet geselecteerd, omdat de keuze is gebaseerd op de gemiddelde penalty en de standaarddeviatie over meerdere runs. We geven daarmee de voorkeur aan parameterinstellingen die gemiddeld goede resultaten opleveren met een relatief beperkte spreiding, in plaats van de instellingen die één keer de laagste penalty opleverden.

**Experiment 3** Instellen van de juiste random seed

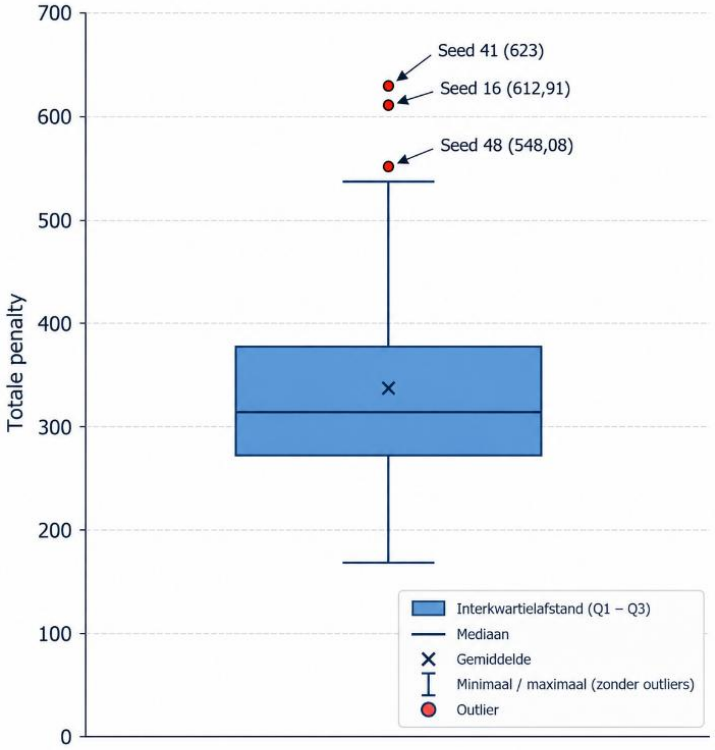
Omdat de gevonden oplossing per random seed kan verschillen, hebben we de eerder bepaalde parameters constant gehouden en de resultaten van 50 verschillende random seeds vergeleken. Vervolgens hebben we de mediaan van de totale penalties bepaald.

We kiezen niet uitsluitend voor de seed met de laagste penalty, omdat deze uitkomst afhankelijk kan zijn van toeval. De mediaan geeft een representatief beeld van de centrale waarde van de gevonden penalties.

In de resultaten zijn drie uitschieters gevonden. Na het uitsluiten van deze uitschieters bedraagt de mediaan van de overgebleven totale penalties 309,28. Random seed 4 levert een penalty op die het dichtst bij deze mediaan ligt en is daarom geselecteerd voor de uiteindelijke planning.

**Figuur 5.7:**

*Verdeling van de totale penalty over 50 random seeds*



**Experiment 4** Parameterinstellingen voor de novemberdataset

Voor de novemberdataset zijn aanvankelijk dezelfde parameters gebruikt als voor de septemberdataset. Dit resulteerde echter in een relatief hoge totale penalty van meer dan 700. Daarom is onderzocht hoe de resultaten verbeterd konden worden. Als eerste is het maximale aantal iteraties t_max verhoogd naar 7.000.000. Dit resulteerde in een lagere totale penalty van 541,97. Door het hogere aantal iteraties krijgt het Simulated Annealing-algoritme meer mogelijkheden om verschillende oplossingen te onderzoeken en een betere oplossing te vinden.

**Tabel 5.1**

*Parameteroptimalisatie voor de novemberdataset*

| t_max | temp | cooling_it | cooling_factor | Random seed | Total penalty | Totale rekentijd (s) |
|---|---|---|---|---|---|---|
| 7000000 | 1000 | 7000 | 0,99541 | 88 | 378,79 | 982,67 |
| 7000000 | 1000 | 7000 | 0,99541 | 1 | 456,08 | 971,91 |
| 700000 | 1000 | 700 | 0,99541 | 88 | 461,48 | 74,65 |
| 7000000 | 1000 | 1400 | 0,99908 | 1 | 477,75 | 753,30 |
| 7000000 | 1000 | 700 | 0,99954 | 1 | 590,29 | 744,37 |
| 7000000 | 1000 | 700 | 0,99954 | 88 | 602,51 | 759,66 |
| 7000000 | 1000 | 1400 | 0,99908 | 88 | 681,86 | 864,96 |
| 700000 | 1000 | 700 | 0,99541 | 1 | 692,10 | 73,92 |
| 700000 | 1000 | 7000 | 0,95499 | 88 | 760,44 | 72,79 |
| 700000 | 1000 | 7000 | 0,95499 | 1 | 856,49 | 72,33 |
| 700000 | 1000 | 1400 | 0,99083 | 88 | 877,65 | 75,56 |
| 700000 | 1000 | 1400 | 0,99083 | 1 | 1443,49 | 75,58 |

Na het uitvoeren van verschillende tests met uiteenlopende parameterinstellingen is een planning gevonden met een totale penalty van 378,79 (**Tabel 5.1**). Dit resultaat is behaald met de volgende parameters: t_max = 7.000.000, temp = 1000, cooling_it = 7000, cooling_factor = 0,99541 en random seed = 88. De rekentijd is aanzienlijk hoger dan bij de septemberdataset. Dit is logisch aangezien de novemberdataset groter is en het algoritme meer iteraties uitvoert.

## 5.3 Vergelijking van de oplosmethoden

### 5.3.1 Experimentele opzet

In dit experiment vergelijken we de prestaties van greedy Heuristic en Simulated Annealing. Hiervoor gebruiken we de PaintShop-dataset van september 2026, waarbij de oorspronkelijke machinesnelheden gelijk blijven. We vergelijken beide methoden zowel vóór als na het toepassen van Discrete Improving Search.

Voor Simulated Annealing gebruiken we de parameterinstellingen uit paragraaf 5.2: 700.000 iteraties, een begintemperatuur van 500 en een temperatuurverlaging van iedere 700 iteraties. De cooling factor wordt zo berekend dat aan het einde nog 1% van de begintemperatuur overblijft.

Omdat Simulated Annealing afhankelijk is van willekeurige keuzes, voeren we zes runs uit met verschillende random seeds: 1, 4, 9, 14, 32 en 88. De Greedy Heuristic voeren we één keer uit, omdat deze deterministisch is.

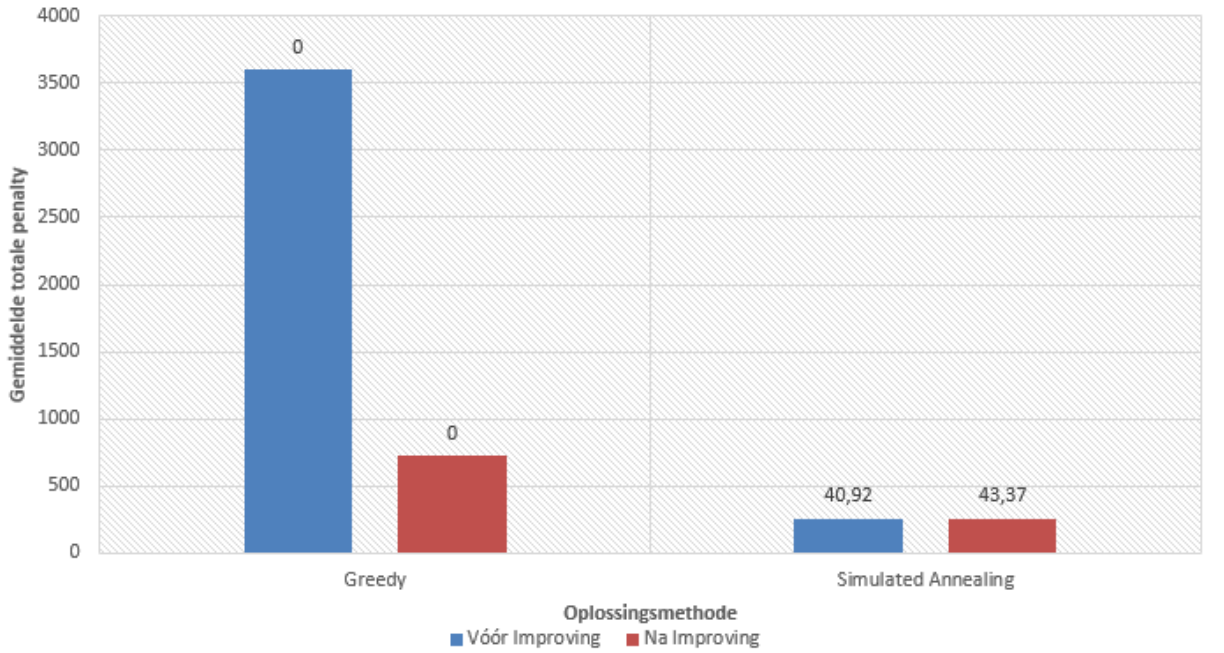
Bij iedere uitvoering vergelijken we de totale penalty, totale tardiness en rekentijd, zowel vóór als na Improving Search. Voor Simulated Annealing berekenen we daarnaast de gemiddelde penalty en standaarddeviatie. Hiermee onderzoeken we welke methode de beste balans biedt tussen oplossingskwaliteit en rekentijd.

### 5.3.2 Vergelijking van de oplossingskwaliteit

Om de oplossingskwaliteit van beide methoden te vergelijken, kijken we naar de gemiddelde totale penalty vóór en na het toepassen van Discrete Improving Search. De resultaten zijn weergegeven in figuur 5.7.

**Figuur 5.7**

*Gemiddelde totale penalty van Greedy en Simulated Annealing vóór en na Discrete Improving Search. De getallen boven de staven van Simulated Annealing geven de standaarddeviatie van de zes runs weer.*




In figuur 5.7 is te zien dat de Greedy Heuristic vóór Improving Search een relatief hoge totale penalty van 3597,75 oplevert. Na het toepassen van Improving Search daalt deze naar 722,14. Dit is een verbetering van ongeveer 80%.

Simulated Annealing behaalt daarentegen al vóór Improving Search een aanzienlijk lagere gemiddelde penalty van 255,98. Na Improving Search daalt deze slechts licht naar 254,31. Dit betekent dat Improving Search bij Simulated Annealing nauwelijks extra verbetering oplevert.

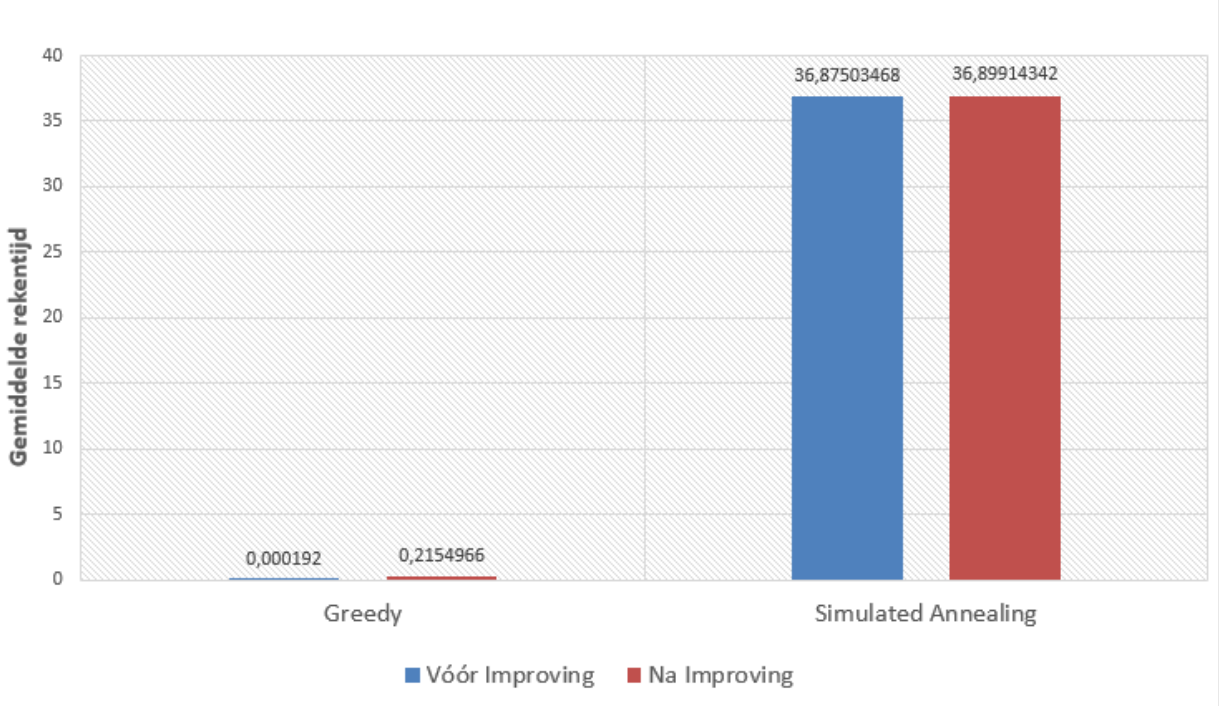
Simulated Annealing is uitgevoerd met zes verschillende random seeds. De standaarddeviatie bedraagt 40,92 vóór Improving Search en 43,37 na Improving Search. Dit laat zien dat de gevonden oplossingen per run verschillen. Opvallend is dat de standaarddeviatie na Improving Search iets toeneemt, terwijl de gemiddelde penalty nauwelijks afneemt. Improving Search zorgt bij Simulated Annealing dus niet voor een kleinere spreiding in de resultaten. Ondanks deze spreiding blijft de gemiddelde penalty aanzienlijk lager dan bij de Greedy Heuristic.

De beste afzonderlijke oplossing van Simulated Annealing werd tijdens dit experiment gevonden bij random seed 1, met een totale penalty van 208,50 vóór Improving Search en 198,50 na Improving Search. Dit laatste resultaat ligt onder de gemiddelde penalty van 254,31 na Improving Search.

### 5.3.3 Vergelijking van de rekentijd

Naast de oplossingskwaliteit vergelijken we ook de benodigde rekentijd van beide methoden. De resultaten zijn weergegeven in figuur 5.8.

*Figuur 5.8: Gemiddelde totale rekentijd van Greedy en Simulated Annealing vóór en na Discrete Improving Search.*




Uit figuur 5.8 blijkt dat de Greedy Heuristic aanzienlijk sneller is dan Simulated Annealing. Greedy heeft slechts 0,0002 seconden nodig om een planning te maken, terwijl Simulated Annealing gemiddeld 36,88 seconden nodig heeft

Na het toepassen van Discrete Improving Search neemt de totale rekentijd van Greedy toe naar 0,22 seconden. Bij Simulated Annealing stijgt deze naar 36,90 seconden. Hieruit blijkt dat Improving Search bij beide methoden relatief weinig extra rekentijd kost, hoewel deze toename bij Greedy in verhouding veel groter is.

Simulated Annealing levert gemiddeld een lagere totale penalty op, maar heeft hiervoor aanzienlijk meer rekentijd nodig. Dit laat zien dat er een duidelijke afweging bestaat tussen de oplossingskwaliteit en de benodigde rekentijd.

### 5.3.4 Toegevoegde waarden van Discrete Improving Search

Om de toegevoegde waarde van Discrete Improving Search te bepalen, vergelijken we de totale penalty en rekentijd vóór en na het toepassen van deze methode.

Bij de Greedy Heuristic daalt de totale penalty van 3597,75 naar 722,14. Dit is een verbetering van ongeveer 80%, terwijl de extra rekentijd slechts 0,22 seconden bedraagt. Discrete Improving Search levert bij Greedy dus een aanzienlijke verbetering op tegen een beperkte extra rekentijd.

Bij Simulated Annealing daalt de gemiddelde penalty van 255,98 naar 254,31. Dit is een verbetering van slechts 0,65%, met een gemiddelde extra rekentijd van ongeveer 0,02 seconden. De verbetering is hier aanzienlijk kleiner, omdat Simulated Annealing zelf al relatief goede oplossingen vindt.

Hieruit blijkt dat Discrete Improving Search vooral bij de Greedy Heuristic een grote toegevoegde waarde heeft. Bij Simulated Annealing is de extra verbetering beperkt, maar ook de benodigde rekentijd is minimaal. Het toepassen van Discrete Improving Search is daarom bij beide methoden mogelijk, maar levert vooral bij Greedy een duidelijk voordeel op.

### 5.3.5 Conclusie

Uit de experimenten blijkt dat Simulated Annealing gemiddeld een betere oplossingskwaliteit levert dan de Greedy Heuristic. Na Discrete Improving Search bedraagt de gemiddelde totale penalty bij Simulated Annealing 254,31, tegenover 722,14 bij Greedy.

Daar staat tegenover dat Simulated Annealing aanzienlijk meer rekentijd nodig heeft. Greedy met Improving Search heeft ongeveer 0,22 seconden nodig, terwijl Simulated Annealing met Improving Search gemiddeld 36,90 seconden duurt bij de gekozen parameterinstellingen.

Discrete Improving Search levert vooral bij Greedy een grote verbetering op. Bij Simulated Annealing is de extra verbetering beperkt, omdat deze methode vóór Improving Search al relatief goede oplossingen vindt.

Op basis van deze resultaten kiezen we voor Simulated Annealing met Discrete Improving Search als uiteindelijke oplossingsmethode. Hoewel deze methode meer rekentijd nodig heeft, weegt de gemiddeld lagere totale penalty voor ons zwaarder.

## 5.4 Keuze van oplosmethode en planning

Op basis van de uitgevoerde experimenten kiezen we voor Simulated Annealing, gevolgd door Discrete Improving Search, als uiteindelijke oplossingsmethode. Uit paragraaf 5.3 blijkt dat deze combinatie gemiddeld een aanzienlijk lagere totale penalty oplevert dan de Greedy Heuristic met Improving Search.

Voor Simulated Annealing gebruiken we de parameterinstellingen die in paragraaf 5.2 zijn bepaald: 700.000 iteraties, een begintemperatuur van 500 en een temperatuurverlaging na iedere 700 iteraties. De cooling factor wordt zo berekend dat aan het einde van het algoritme nog 1% van de begintemperatuur overblijft. Voor de uiteindelijke planning gebruiken we random seed 4, die in het eerdere onderzoek als representatieve seed is geselecteerd.

De parameterinstellingen zijn gekozen op basis van een lage gemiddelde penalty en een relatief kleine standaarddeviatie, in plaats van uitsluitend de beste afzonderlijke uitkomst. Met deze methode en parameterinstellingen bepalen we de uiteindelijke planning voor de PaintShop-dataset van september 2026.

# 6 Eigen onderzoeksvraag of ontwikkelrichting

In dit hoofdstuk werken wij aan een eigen, relevante onderzoekvraag of ontwikkelrichting systematisch. We laten zien waarom deze uitbreiding zinvol is, hoe deze is onderzocht en wat de toegevoegde waarde ervan is.

## 6.1 Onderzoeksvraag of ontwikkelrichting

Onze onderzoeksvraag luidt:

*Welke combinatie van machinesnelheidsverhogingen is nodig om een totale penalty van 0 te bereiken met een zo klein mogelijke totale snelheidsverhoging, en in hoeverre verschilt dit tussen de twee oplossingsmethoden?*

We onderzoeken welke invloed veranderingen in de machinesnelheden hebben op de totale penalty door de machinesnelheden zowel afzonderlijk als in verschillende combinaties te verhogen. Hierbij vergelijken we twee oplossingsmethoden:

**1. Greedy → Improving Search**: Eerst wordt een planning opgesteld met de Greedy-methode, waarna deze wordt verbeterd met Improving Search , waarbij de Best Improvement-zoekstrategie wordt toegepast.

**2. Metaheuristiek → Improving Search**: Eerst wordt een planning opgesteld met de metaheuristiek, waarna deze verder wordt verbeterd met Improving Search , waarbij de Best Improvement-zoekstrategie wordt toegepast.

Vanwege de beperkte beschikbare tijd richten we ons uitsluitend op de dataset van september. De dataset van november wordt daarom buiten beschouwing gelaten.

Het doel is om te bepalen met welke combinatie van machinesnelheidsverhogingen een totale penalty van 0 kan worden bereikt, waarbij de som van de procentuele snelheidsverhogingen zo klein mogelijk blijft. Daarnaast onderzoeken we in hoeverre de gevonden resultaten verschillen tussen beide oplossingsmethoden.

## 6.2 Aanpak

Het uiteindelijke doel van de planning is om de totale penalty zo laag mogelijk te krijgen, bij voorkeur tot nul. In hoofdstuk 5 hebben we onderzocht welke parameterinstellingen voor Simulated Annealing goede en consistente resultaten opleveren. In dit hoofdstuk onderzoeken we welke invloed het verhogen van de machinesnelheden heeft op de totale penalty. Hierbij willen we bepalen welke machines sneller moeten draaien om alle orders zonder vertraging af te ronden, met een zo klein mogelijke totale snelheidsverhoging.

Voor dit onderzoek gebruiken we de PaintShop-dataset van september 2026. De oorspronkelijke machinesnelheden en bijbehorende totale penalty dienen als referentiewaarde.

We onderzoeken eerst de machines afzonderlijk door telkens de snelheid van één machine met 10%, 20%, 30% en 50% te verhogen, terwijl de andere snelheden gelijk blijven. Hiermee bepalen we welke machine de grootste invloed heeft op de totale penalty.

Vervolgens onderzoeken we verschillende combinaties van snelheidsverhogingen. Hierbij testen we onder andere gelijke verhogingen voor alle drie de machines. Op basis van deze resultaten onderzoeken we kleinere intervallen en verschillende combinaties van snelheidsverhogingen om te bepalen bij welke combinatie voor het eerst een penalty van 0 wordt gevonden.

Bij ieder experiment maken we eerst een planning met de Greedy Heuristic, gevolgd door Discrete Improving Search. De totale penalty dient als belangrijkste prestatiemaat en wordt vergeleken met de oorspronkelijke situatie.

Daarnaast vergelijken we deze resultaten met Simulated Annealing, eveneens gevolgd door Discrete Improving Search. Hiervoor gebruiken we de parameterinstellingen uit hoofdstuk 5. Vanwege het grote aantal combinaties en de benodigde rekentijd verlagen we het maximale aantal iteraties aanvankelijk naar 70.000. De begintemperatuur blijft 500 en de temperatuur wordt na iedere 700 iteraties verlaagd. Voor het nauwkeurigere onderzoek verhogen we het maximale aantal iteraties naar 700.000.

Om de experimenten reproduceerbaar en vergelijkbaar te maken, gebruiken we random seed 4. Deze is in hoofdstuk 5 geselecteerd omdat de bijbehorende penalty het dichtst bij de mediaan lag.

Door de experimenten stapsgewijs uit te voeren, onderzoeken we welke combinatie van machinesnelheden een penalty van 0 oplevert met een zo klein mogelijke totale snelheidsverhoging in procenten.

## 6.3 Resultaten

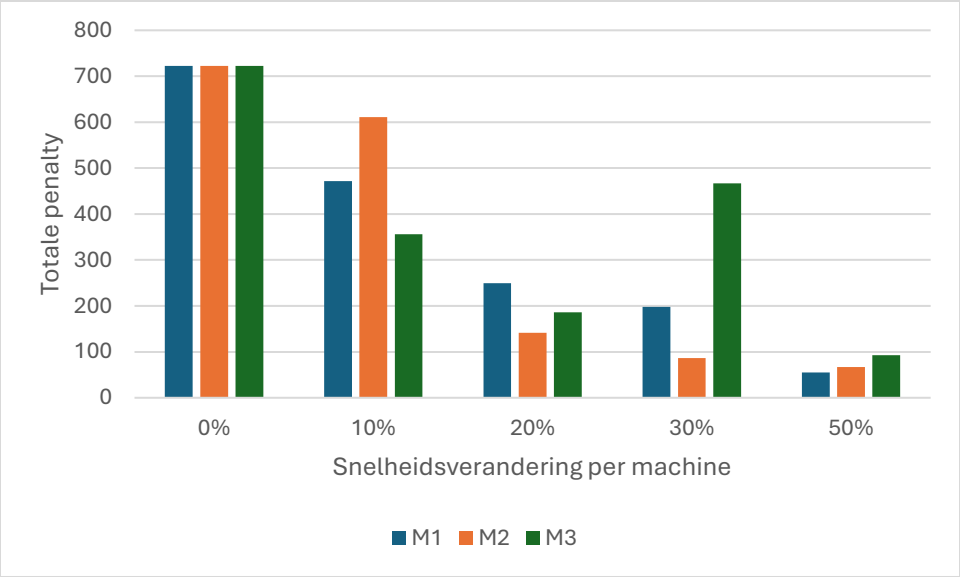
**Onderzoek 1:**

Eerst onderzoeken we de invloed van veranderingen in de machinesnelheden op de totale penalty met behulp van de Greedy-methode gevolgd door Improving Search. Dit onderzoek bestaat uit meerdere stappen, die hieronder worden toegelicht.

**Stap 1:** De snelheid van één machine afzonderlijk met verschillende percentages verhogen**.**

**Figuur 6.1**

*Invloed van veranderingen in machinesnelheid op de totale penalty*




In **figuur 6.1** is te zien dat Een snelheidsverhoging van 10% leidt bij alle drie de machines tot een daling van de totale penalty. Naarmate de machinesnelheid verder wordt verhoogd, neemt de penalty over het algemeen verder af. Bij een snelheidsverhoging van 50% is de totale penalty voor alle drie de machines aanzienlijk gedaald ten opzichte van de oorspronkelijke situatie.

**Stap 2:** Het effect van verschillende combinaties van snelheidsverhogingen onderzoeken.

Onze vervolgstap is om verschillende combinaties van snelheidsverhogingen te onderzoeken en te bepalen welke combinatie de laagste totale penalty oplevert. Alleen de beste resultaten zijn weergegeven in **tabel 6.1**.

**Tabel 6.1**

*Invloed van gecombineerde machinesnelheidsverhogingen op de totale penalty*

| Combinatie | M1 (%) | M2 (%) | M3 (%) | Totale penalty | Verbetering (%) |
|---|---|---|---|---|---|
| Geen verhoging | 0 | 0 | 0 | 722,1 | 0,0 |
| M2 + M3 | 0 | 20 | 50 | 17,0 | 97,6 |
| M1 + M3 | 50 | 0 | 50 | 8,8 | 98,8 |
| **M1 + M2** | **50** | **50** | **0** | **0,0** | **100,0** |
| **M1 + M2 + M3** | **30** | **30** | **30** | **0,0** | **100,0** |

In **tabel 6.1** is te zien dat twee combinaties een totale penalty van 0 opleveren. Opvallend is dat de combinatie waarbij alle drie de machines 30% sneller draaien, hetzelfde resultaat behaalt als de combinatie waarbij M1 en M2 beide 50% sneller draaien. De som van de procentuele snelheidsverhogingen is bij de eerste combinatie echter lager (90% tegenover 100%). Dit laat zien dat het verhogen van de snelheid van alle drie de machines effectiever kan zijn dan het sterk verhogen van slechts twee machines.

**Stap 3:** Onderzoeken binnen welk interval er een nuloplossing kan worden gevonden**.**

**Tabel 6.2**

*Invloed van gelijke snelheidsverhogingen op de totale penalty na Improving Search*

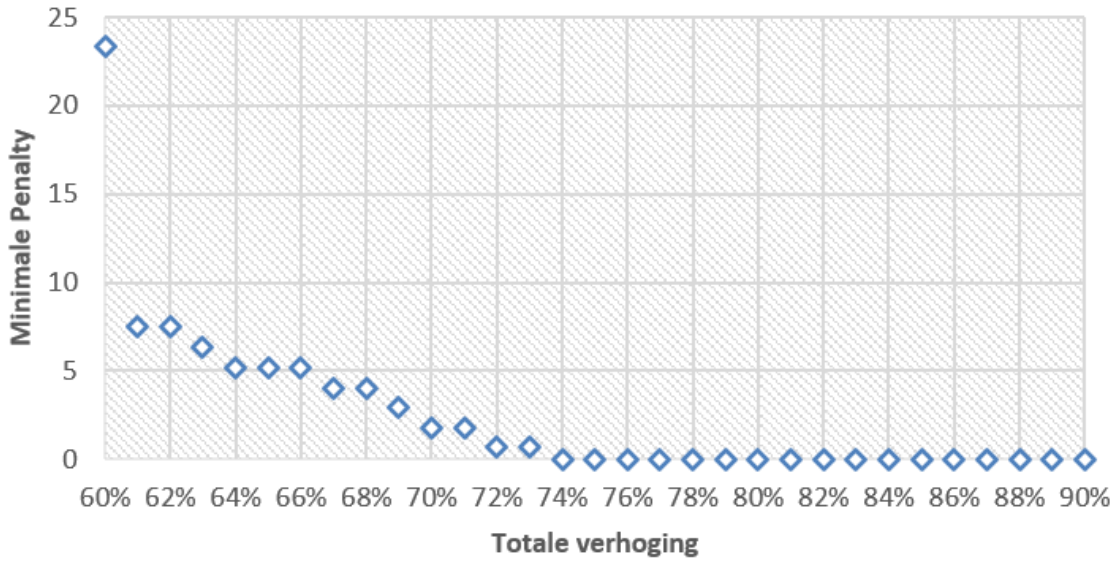
| M1 (%) | M2 (%) | M3 (%) | Totale penalty | Verbetering (%) |
|---|---|---|---|---|
| **30** | **30** | **30** | **0,0** | **100,0** |
| 25 | 25 | 25 | 1,8 | 99,8 |
| 20 | 20 | 20 | 23,3 | 96,8 |
| 15 | 15 | 15 | 181,5 | 74,9 |
| 10 | 10 | 10 | 206,1 | 71,5 |
| 5 | 5 | 5 | 485,8 | 32,7 |
| 0 | 0 | 0 | 722,1 | 0,0 |

In **tabel 6.2** is te zien dat bij snelheidsverhogingen onder de 20% een aanzienlijk hogere totale penalty optreedt. Onze vervolgstap is daarom om snelheidsverhogingen tussen de 20% en 30% nader te onderzoeken en te bepalen bij welke percentages de laagste penalty wordt behaald. Door kleinere stappen te gebruiken kunnen we nauwkeuriger bepalen bij welke combinatie de penalty voor het eerst 0 wordt.

**Stap 4:** Het gevonden interval nader onderzoeken om het mogelijke optimum te bepalen.

**Figuur 6.2**

*Minimale penalty bij toenemende totale snelheidsverhoging*



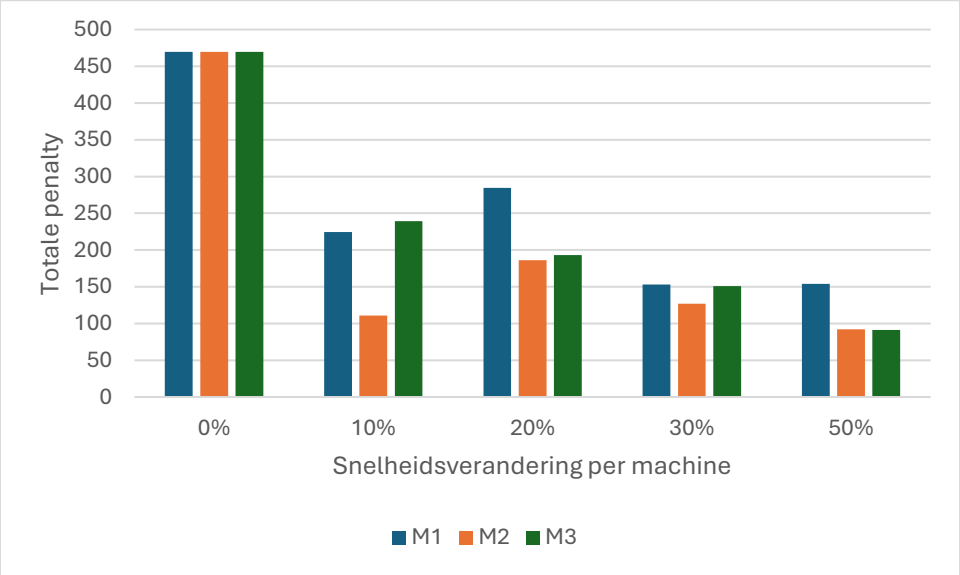

In **figuur 6.2** is te zien dat bij een totale snelheidsverhoging van 74% voor het eerst een penalty van 0 wordt bereikt. Dit is de kleinste totale snelheidsverhoging waarbij binnen de onderzochte combinaties een penalty van 0 is gevonden. De bijbehorende snelheidsverhogingen zijn 27% voor M1, 27% voor M2 en 20% voor M3. Hiermee wordt niet alleen een penalty van 0 bereikt, maar ook de totale benodigde snelheidsverhoging zo klein mogelijk gehouden binnen de onderzochte combinaties.

**Onderzoek 2:**

Als tweede onderzoeken we de invloed van veranderingen in de machinesnelheden op de totale penalty met behulp van de metaheuristiek, gevolgd door Improving Search. Ook dit onderzoek wordt in meerdere stappen uitgevoerd, die hieronder verder worden toegelicht. We zijn begonnen met een verkennend onderzoek, waarbij het maximale aantal iteraties op 70.000 is ingesteld.

**Stap 1:** De snelheid van één machine telkens met 10% verhogen.

**Figuur 6.3**
Invloed van veranderingen in machinesnelheid op de totale penalty




In **figuur 6.3** is te zien dat een verhoging van de machinesnelheid over het algemeen leidt tot een lagere totale penalty. Ondanks enkele schommelingen neemt de penalty bij verdere snelheidsverhogingen af. Bij een verhoging van 50% is de totale penalty voor alle drie de machines aanzienlijk lager dan in de oorspronkelijke situatie.

**Stap 2:** Het effect van verschillende combinaties van snelheidsverhogingen onderzoeken.

Er is een vergelijkbaar patroon zichtbaar als bij de Greedy-methode met Improving Search. Aangezien het uiteindelijke doel is om een totale penalty van 0 te behalen, stellen we het maximale aantal iteraties in op 700.000. Uit **figuur 6.2** blijkt dat bij de Greedy-methode een totale snelheidsverhoging van 74% nodig was om een penalty van 0 te bereiken. Daarom richten we ons bij dit experiment op snelheidsverhogingen rond dit percentage. Omdat Simulated Annealing, zoals beschreven in paragraaf 5.3.5, doorgaans betere oplossingen oplevert dan de Greedy-methode, verwachten we dat met deze methode mogelijk een lagere totale snelheidsverhoging voldoende is om een penalty van 0 te behalen.

**Tabel 6.3**

*Resultaten bij gelijke snelheidsverhogingen per machine*

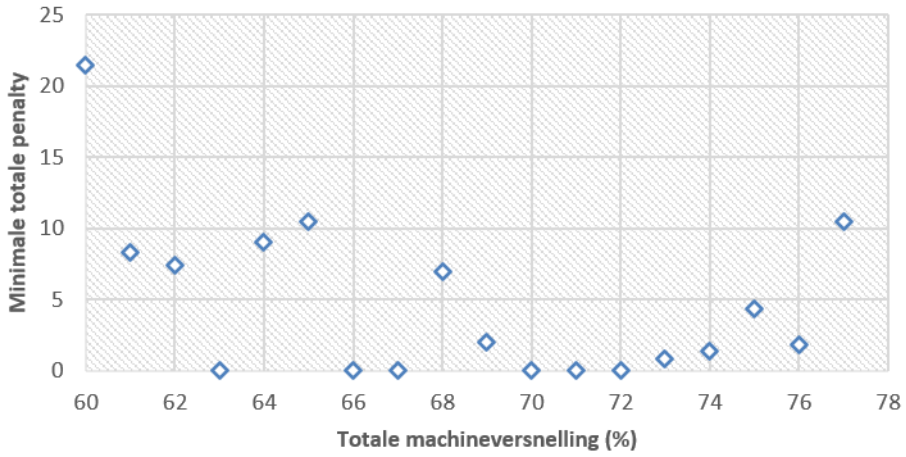
| M1 (%) | M2 (%) | M3 (%) | Penalty SA Improved | Rekentijd SA (s) |
|---|---|---|---|---|
| 25 | 25 | 25 | 6,00 | 35,44 |
| 26 | 26 | 26 | 8,33 | 35,28 |
| 27 | 27 | 27 | 9,27 | 35,46 |
| 28 | 28 | 28 | 0,00 | 35,57 |
| 29 | 29 | 29 | 0,00 | 35,47 |
| 30 | 30 | 30 | 0 | 35,92 |

In **tabel 6.3** zijn de resultaten weergegeven van verschillende gelijke snelheidsverhogingen voor de drie machines. Voor iedere combinatie zijn dezelfde SA-instellingen gebruikt, waardoor de rekentijden vergelijkbaar zijn. Uit de resultaten blijkt dat bij een snelheidsverhoging van 28% per machine voor het eerst een totale penalty van 0 wordt behaald. Bij de Greedy-methode was hiervoor een verhoging van 30% per machine nodig.

**Stap 3:** De machinesnelheden optimaliseren

**Figuur 6.4**

*Minimale penalty per totale snelheidsverhoging*




Zoals te zien is in **figuur 6.4**, is er geen duidelijke dalende trend zichtbaar, in tegenstelling tot **figuur 6.2** bij de Greedy-methode. Dit kan worden verklaard door het willekeurige karakter van Simulated Annealing, waardoor de gevonden oplossingen per experiment kunnen verschillen. Daarnaast blijkt dat bij een totale machinesnelheidsverhoging van slechts 63% al een penalty van 0 wordt behaald. Hierbij zijn de snelheidsverhogingen voor M1, M2 en M3 respectievelijk 20%, 20% en 23%.

## 6.4 Conclusie en toegevoegde waarde

In het hierboven uitgevoerde onderzoek stond de volgende onderzoeksvraag centraal:

*Welke combinatie van machinesnelheidsverhogingen is nodig om een totale penalty van 0 te bereiken met een zo klein mogelijke totale snelheidsverhoging, en in hoeverre verschilt dit tussen de twee oplossingsmethoden?*

Uit de resultaten in paragraaf 6.3 blijkt dat het verhogen van de machinesnelheden over het algemeen leidt tot een lagere totale penalty. Bij de Greedy-methode met Improving Search is een penalty van 0 gevonden bij snelheidsverhogingen van 27% voor M1, 27% voor M2 en 20% voor M3. De som van deze verhogingen bedraagt 74 procentpunten.

Met Simulated Annealing en Improving Search is binnen de onderzochte combinaties een penalty van 0 gevonden bij snelheidsverhogingen van 20% voor M1, 20% voor M2 en 23% voor M3. De som hiervan bedraagt 63 procentpunten. Hiermee werd binnen de uitgevoerde experimenten een nuloplossing gevonden bij een kleinere som van snelheidsverhogingen dan met de Greedy-methode.

Het onderzoek laat zien dat het aanpassen van de machinesnelheden kan leiden tot een aanzienlijk lagere totale penalty. Het voordeel van Greedy met Improving Search is dat deze methode aanzienlijk minder rekentijd nodig heeft dan Simulated Annealing. SA met Improving Search behaalde daarentegen binnen de uitgevoerde experimenten een nuloplossing bij een kleinere som van snelheidsverhogingen.

Omdat Simulated Annealing afhankelijk is van willekeurige keuzes, kunnen de resultaten per uitvoering verschillen. In dit onderzoek is uitsluitend random seed 4 gebruikt. Hierdoor zijn de experimenten reproduceerbaar, maar kan niet worden vastgesteld of dezelfde resultaten ook bij andere random seeds worden behaald.

Het combineren van planningsoptimalisatie met hogere machinesnelheden kan een waardevolle aanpak zijn om de totale penalty van een productieproces te verlagen. Wel kunnen hogere machinesnelheden leiden tot meer energieverbruik, slijtage en mogelijk kwaliteitsverlies. Daarnaast is uitsluitend de septemberdataset onderzocht, waardoor niet zeker is of de resultaten ook voor andere maanden gelden.

# 7 Conclusie

In dit project zijn twee oplossingsmethoden onderzocht om de totale penalty van een PaintShop-planning te minimaliseren: de Greedy EDD-heuristiek en Simulated Annealing (SA), beide gevolgd door Discrete Improving Search op basis van swaps. Hiermee wordt een lokaal optimum bereikt waarbij geen enkele swap binnen de onderzochte neighbourhood de planning verder kan verbeteren. Voor beide methoden zijn Python-scripts ontwikkeld waarmee planningen kunnen worden weergegeven in een Gantt-chart (Figuur 3.1) en geëxporteerd naar een Excel-bestand. De implementaties zijn gevalideerd aan de hand van 13 geslaagde tests, waarbij berekeningen zowel handmatig als met Python zijn gecontroleerd. De tests en experimenten zijn uitgevoerd met de septemberdataset, met uitzondering van experiment 4, waarin de parameters voor de novemberdataset zijn onderzocht.

Uit de complexiteitsanalyses blijkt dat de tijdscomplexiteit van de oplossingsmethoden onder andere afhangt van het aantal orders. Improving Search onderzoekt per verbeteringsronde $O(n^2)$ mogelijke swaps, terwijl SA per iteratie één mogelijke swap onderzoekt. Door het grote aantal iteraties heeft SA echter een langere totale rekentijd. Greedy EDD met Improving Search behaalde een penalty van 722,14 in ongeveer 0,22 seconden, tegenover een gemiddelde penalty van 255,98 voor SA en 254,31 na Improving Search, bij een totale rekentijd van ongeveer 36,90 seconden. Tijdens het parameteronderzoek met de septemberdataset werd een laagste afzonderlijke penalty van 146,06 gevonden. Voor de uiteindelijke methode zijn 700.000 iteraties, een begintemperatuur van 500 en een temperatuurverlaging na iedere 700 iteraties gekozen, met een cooling factor waarbij uiteindelijk 1% van de begintemperatuur overblijft. Random seed 4 is geselecteerd omdat de bijbehorende penalty het dichtst bij de mediaan lag. Deze keuzes zijn gebaseerd op een lage gemiddelde penalty en een relatief kleine standaarddeviatie, in plaats van uitsluitend de beste afzonderlijke uitkomst. Ondanks de langere rekentijd heeft SA met Improving Search de voorkeur vanwege de gemiddeld lagere penalty. Voor de novemberdataset zijn afzonderlijke parameters onderzocht en is een penalty van 378,79 gevonden.

Tot slot is onderzocht in hoeverre het verhogen van de machinesnelheden kan bijdragen aan het bereiken van een penalty van 0. Dit bleek mogelijk met beide oplossingsmethoden. In de onderzochte combinaties werd met Greedy EDD en Improving Search een penalty van 0 bereikt bij afzonderlijke snelheidsverhogingen waarvan de percentages samen 74 procentpunt bedragen, tegenover 63 procentpunt bij SA met Improving Search. Deze waarden vertegenwoordigen niet de procentuele toename van de gezamenlijke machinecapaciteit. Het onderzoek laat zien dat het combineren van

planningsoptimalisatie met hogere machinesnelheden de totale penalty verder kan verlagen. Daarbij moet rekening worden gehouden met de praktische haalbaarheid, de extra kosten en de invloed van willekeurige keuzes bij SA.

# References


Auger, N., Jugé, V., Nicaud, C., & Pivoteau, C. (2018). On the worst-case complexity of TimSort. Leibniz International Proceedings in Informatics (LIPIcs), 112, Article 4. https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2018.4

Dong, X., Nowak, M., Chen, P., & Lin, Y. (2015). Self-adaptive perturbation and multi-neighborhood search for iterated local search on the permutation flow shop problem. Computers & Industrial Engineering, 87, 176–185. https://www.sciencedirect.com/science/article/pii/S0360835215002089?via%3Dihub

Idzikowski, R., Rudy, J., & Gnatowski, A. (2021). Solving non-permutation flow shop scheduling problem with time couplings. Applied Sciences, 11(10), Article 4425. https://www.mdpi.com/2076-3417/11/10/4425

Massachusetts Institute of Technology. (z.d.). Big-O notation and recurrence equations [Cursusmateriaal]. MIT OpenCourseWare. https://ocw.mit.edu/courses/16-01-unified-engineering-i-ii-iii-iv-fall-2005-spring-2006/9ea490a9e30d004f9a24f4151b05749b_bigorecurrences.pdf

Zhang, Z., Zhao, W., Zhao, H., & Bian, X. (2025). MNATS: A multi-neighborhood adaptive tabu search algorithm for the distributed no-wait flow shop scheduling problem. Applied Sciences, 15(17), Article 9840. https://www.mdpi.com/2076-3417/15/17/9840

# Bijlage A: Documentatie gebruik AI

Tijdens het uitvoeren van dit project hebben we gebruikgemaakt van ChatGPT als ondersteunend hulpmiddel. We hebben AI voornamelijk ingezet voor het verduidelijken van programmeerconcepten, het oplossen van foutmeldingen, het ondersteunen bij experimenten en het verbeteren van de formulering van ons verslag.

We blijven zelf verantwoordelijk voor de ingeleverde code, de gemaakte keuzes en de conclusies in het verslag. AI-gegenereerde suggesties zijn niet zonder controle overgenomen.

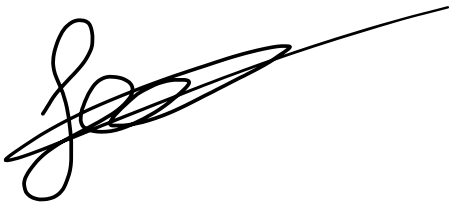
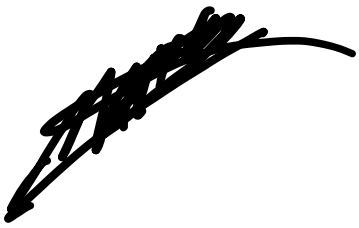
**Verantwoordelijkheidsverklaring**

Wij verklaren verantwoordelijk te zijn voor het volledige ingeleverde werk en de gebruikte oplossingsmethoden, experimenten en conclusies te kunnen toelichten.

Namen:
Fee van Zoghel & Matthijs van Meeuwen

Datum:
02-11-2026

Handtekeningen:

| Fee van Zoghel | Matthijs van Meeuwen |
| --- | --- |
|  |  |

# Bijlage B: Checklist Python Code

The checklist helps the assessor evaluate the REALISE criterion efficiently. It does not replace validation of correctness. By completing this checklist, you confirm that the references given above accurately describe the submitted implementation.

Complete this checklist before submission. For each item, indicate whether the requirement is fully met, partly met, or not met. In the last column, point to the relevant evidence in your submitted Python code, for example by giving a file name and function name.

| Item | Requirement | Fully met? | Evidence in submitted code |
|---|---|---|---|
| A1 | The implemented logic for the greedy constructive heuristic is consistent with the pseudocode and documented design choices. | Yes | Bestand: Greedy.py<br>Functies: greedy_schedule() en calculate_results() |
| A2 | The implemented logic for the metaheuristic is consistent with the pseudocode and documented design choices. | Yes | Bestand: Metaheuristic.py<br>Functies: SA() en calculate_tard_pen() |
| A3 | The neighbourhood structure, move(s), move-selection strategy and termination logic can be easily identified in the Python code. | Yes | Bestand: Improved_greedy.py<br>Functie: improving_search()<br><br>Bestand: Improved_metaheuristic.py<br>Functie: meta_improving_search()<br><br>Bestand: Functions.py<br>Functies: random_swap() en swap() |
| B1 | The Python implementation imports the prescribed Excel input data. | Yes | Bestanden: Greedy.py en Metaheuristic.py<br>Functies: pd.read_excel(), df[‘Orders’], df[‘Machines’], df[‘Setups’] |
| B2 | The same implementation can process both supplied problem instances. | Yes | Bestanden: Greedy.py en Metaheuristic.py<br>Functies: DATA_DIR en pd.read_excel(); len(di_orders) en len(di_machines) |
| B3 | The Python implementation generates the required Excel schedule in the prescribed format. | Yes | Bestand: Functions.py<br>Functies: resultaten_naar_excel() en resultaten_naar_excel_list() |

| | | | |
|---|---|---|---|
| B4 | The Python implementation can generate an appropriate schedule visualisation, such as a Gantt chart. | Yes | Bestand: Functions.py<br>Functies: gantt_chart() en gantt_chart_list() |
| B5 | The Python implementation provides an appropriate way to present or visualise computational progress where relevant. | Yes | Bestand: Improved_metaheuristic.py<br>Functies: plot_sa_progress() en voortgangsweergave in SA() |